drones

MDPI

Article

A Multi-Swarm Dynamic Crow Search Algorithm for Multi-UAV Dynamic Task Allocation

Gengsong Li, Yi Liu *, Qibin Zheng and Kun Liu

Academy of Military Sciences, Beijing 100091, China; gus_li@163.com (G.L.); zhengqibin1990@163.com (Q.Z.); lauk@pku.edu.cn (K.L.)

* Correspondence: albertliu20th@163.com

Highlights

What are the main findings?

- A dynamic multi-objective optimization model is formulated for multi-UAV dynamic task allocation, incorporating state inheritance and diverse dynamic events.
- A multi-swarm dynamic crow search algorithm with tailed strategies is proposed to solve the problem efficiently.

What is the implication of the main finding?

- The model enables dynamic task allocation that reflects the continuous scenario information and covers a broader range of real-world events.
- The algorithm achieves superior performance with acceptable runtime, shows effectiveness in handling dynamic task allocation problems.

Abstract

Efficient cooperative task allocation is essential for multiple unmanned aerial vehicles (UAVs) performing complex missions. However, diverse dynamic events in real-world scenarios require rapid response through dynamic task allocation (DTA). Although evolutionary algorithms have been widely adopted for DTA, existing methods often fail to maintain consistency between allocation decisions and actual operational states, consider only limited classes of dynamic events, and still leave room for performance improvement. This paper formulates multi-UAV DTA as a dynamic multi-objective optimization problem (DMOP) that jointly minimizes the residual target value and mission makespan, incorporating a state inheritance mechanism and a comprehensive set of dynamic events covering multiple facets of disruptions in observation task scenarios. To solve this DMOP, a multi-swarm dynamic crow search algorithm for task allocation (MDCSATA) is proposed, which integrates five strategies: violation-tolerant multi-swarm co-evolution for feasibility and diversity; objective-oriented heuristic initialization to accelerate convergence; an adaptive position update for better exploration and exploitation; stagnation and elite guided perturbation for intensified local exploitation; and an event-aware change response for rapid adaptation to dynamic events. Experiments on three constructed scenarios against seven state-of-the-art algorithms show that MDCSATA achieves superior performance on the evaluation metrics with acceptable runtime. It obtains the best MHV and MIGD in all scenarios, improving MHV by at least 0.93% and reducing MIGD by at least 12.92% across scenarios. These results confirm its effectiveness for DTA.

Keywords: multi-UAV dynamic task allocation; dynamic multi-objective optimization; crow search algorithm; multi-swarm co-evolution

Check for updates

Academic Editors: Emanuele Luigi de Angelis, Fabrizio Giulietti and Giulio Avanzini

Received: 7 May 2026
Revised: 13 June 2026
Accepted: 16 June 2026
Published: 18 June 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Unmanned aerial vehicles (UAVs) have been widely adopted across various domains, such as remote sensing, plant protection, and damage detection, owing to their flexibility and low operational cost [1–3]. Research has increasingly shifted from single-UAV platforms to multi-UAV cooperative swarms, which can undertake complex tasks beyond the capability of individual units—examples include search and rescue, area surveillance, and supplies delivery [4–6]. Compared with a single-UAV system, a multi-UAV swarm offers distinct advantages in mission efficiency and robustness, because it can execute tasks in parallel and maintain overall mission integrity even if some UAVs are lost [7]. To fully realize these advantages, efficient cooperative task allocation is essential: it assigns specific tasks to individual UAVs according to decision demands and governs how resources are distributed to meet mission objectives, which has made it an extensively studied topic [8–10]. In practice, decision-makers often seek plans that balance diverse operational priorities, for example, maximizing mission reward while minimizing UAV cost. These multi-faceted requirements turn task allocation into a multi-objective optimization problem that requires simultaneous optimization of conflicting objectives [11].

In real-world deployments, however, dynamic events exist that change the scenario information, giving rise to the dynamic task allocation (DTA) problem. Traditional static allocation methods lack the flexibility to adapt to such changes [12]. To address DTA, researchers have proposed heuristic-based, machine learning-based, market mechanism-based, and evolutionary algorithm (EA)-based methods [13–16]. The first three categories mainly rely on problem-specific rules, negotiation, or learned policies, whereas EAs provide flexible search for complex multi-objective allocation problems. Among these, EA-based approaches are widely favored for their scalability, global search capability, and multi-objective optimization performance. For instance, Deng et al. [17] modeled heterogeneous UAV task allocation within a digital twin framework and proposed an improved dynamic multi-objective adaptive weight particle swarm optimization (DMOAWPSO) algorithm, which quantifies environmental severity and adaptively selects population restructuring strategies to track the moving Pareto front (PF). Zhang et al. [18] formulated search and track missions as dynamic multi-objective problems (DMOPs) with varying dimensions and objectives, and proposed the dynamic constrained two-archive evolutionary algorithm (DCTAEA) with a self-adaptive penalty mechanism to reuse information from infeasible solutions. Wang et al. [19] addressed reconnaissance task allocation with moving targets via the dynamic multi-objective particle swarm optimization algorithm based on centroid-moving prediction (DCMPSO), which uses the movement of the population centroid in the decision space between environments to predict the PF location. Moreover, some studies address the DTA problem by combining multiple objectives into one through linear weighting. Zhang et al. [16] modeled forest fire rescue assignment as a dynamic optimization problem driven by stochastic fire spread and proposed a variable-length particle swarm optimization with a dynamic encoding scheme that maps particles to evolving fire-edge cells, enabling adaptive reallocation within a simulation-integrated ensemble framework. Huang et al. [20] established a fuzzy multi-constraint model and employed a four-dimensional information grey wolf optimizer with dynamic time slices to handle environmental uncertainties.

Despite these advances, existing EA-based research still exhibits three major shortcomings. First, most models fail to account for the state continuity of mission execution: they overlook how ongoing tasks continuously alter UAV or task states, leading to a decoupling between decision information and the working environment. Second, the dynamic events considered are relatively narrow in scope, typically focusing only on structural changes such as platform failures or new target appearances, while neglecting events such

as sensor damage, weather-induced capability fluctuations, and decision variations. Third, there still remains room for improvement of the methods in both event response speed and allocation performance. Therefore, existing methods do not simultaneously address execution-state consistency, heterogeneous dynamic events, and rapid adaptation in the constrained multi-UAV DTA scenario considered in this study.

To address these challenges, this paper proposes the multi-swarm dynamic crow search algorithm for task allocation (MDCSATA). On the modeling side, critical information such as UAV coordinates and task completion status is inherited from actual execution results, enhancing the consistency of the dynamic allocation scheme. Diverse dynamic events are introduced, covering UAVs, targets, environment, and decision preferences, constructing a more realistic and comprehensive task allocation scenario. On the algorithmic side, considering the key components of dynamic multi-objective evolutionary algorithms (DMOEAs)—the static optimizer and the change response strategy [21]—we adopt the crow search algorithm (CSA) for its simple search pattern, low parameter count, and competitive optimization performance [22,23]. For change response, event severity is quantified to guide the rapid response to the dynamic events, thereby improving convergence and speed in handling DTA problems. Our main contributions are:

(1) We formulate a DMOP framework for multi-UAV dynamic task allocation. By establishing a state inheritance mechanism and introducing diverse dynamic events, this framework can better mirror real-world operational complexities.
(2) We propose MDCSATA with five coordinated strategies tailored to DTA problems: violation-tolerant multi-swarm co-evolution, objective-oriented heuristic initialization, adaptive position update, stagnation and elite guided perturbation, and event-aware change response.
(3) Three benchmark scenarios with diverse dynamic events are constructed, and MDCSATA is evaluated against seven state-of-the-art DMOEAs using two performance metrics. Experimental results verify that MDCSATA generally achieves superior task allocation performance with acceptable runtime overhead.

The remainder of this paper is organized as follows. Section 2 reviews the background of DMOP and the CSA. Section 3 formulates the multi-UAV DTA problem. Section 4 introduces the proposed MDCSATA algorithm. Section 5 reports the experimental evaluation. Finally, Section 6 concludes the paper and discusses future research directions.

2. Background

2.1. Dynamic Multi-Objective Optimization Problems

Without loss of generality, a minimization DMOP can be defined as

$$\begin{cases} \min\limits_{\boldsymbol{x}} \boldsymbol{F}(\boldsymbol{x},t) = (f_1(\boldsymbol{x},t), f_2(\boldsymbol{x},t), \cdots, f_{M_t}(\boldsymbol{x},t)) \\ \text{s. t. } g_i(\boldsymbol{x},t) \geq 0, i = 1, \cdots, n_g(t) \\ \quad\quad h_j(\boldsymbol{x},t) = 0, j = 1, \cdots, n_h(t) \\ \quad\quad \boldsymbol{x} \in \Omega_x, t \in \Omega_t \end{cases} \tag{1}$$

where $\boldsymbol{x}$ is a decision vector and $\Omega_x \subseteq \mathbb{R}^n$, t is the time variable and $\Omega_t \subseteq \mathbb{R}$. At time t, $\boldsymbol{F}(\boldsymbol{x},t)$ is the objective function vector that evaluates the decision vector $\boldsymbol{x}$, M_t is the number of objectives, $g_i(\boldsymbol{x},t)$ and $h_j(\boldsymbol{x},t)$ are inequality constraints and equality constraints, and $n_g(t)$ and $n_h(t)$ are the numbers of inequality constraints and equality constraints, respectively. A DMOP can be regarded as a time series of static multi-objective optimization problems. Therefore, solving DMOPs involves tracking the Pareto set (PS) and PF of the

multi-objective optimization problem series, and the PS and PF of a DMOP at time t are denoted as PS_t and PF_t.

2.2. Crow Search Algorithm

CSA is inspired by the intelligent behavior of crows that store their own food at secret places and also search for food hidden by other birds to steal it [24]. Owing to its simple search mechanism and computational efficiency, CSA has been extended to various optimization problems [25–27]. CSA assumes each crow's current position is $\boldsymbol{x}$, and each crow hides its food at its best position so far as $\boldsymbol{m}$. At iteration τ, crow i targets crow j to attempt to steal its food. Then, the position update of crow i depends on the awareness probability AP and the flight length fl of crows as follows:

$$\boldsymbol{x}_i^{\tau+1} = \begin{cases} \boldsymbol{x}_i^{\tau} + r_i \times fl \times \left(\boldsymbol{m}_j^{\tau} - \boldsymbol{x}_i^{\tau}\right), & r_j \geq AP \\ \text{a random position}, & r_j < AP \end{cases} \tag{2}$$

where r_i and r_j are independent random numbers with uniform distributions between 0 and 1. In CSA, the memory $\boldsymbol{m}_j^{\tau}$ represents the best food-hiding position found by crow j. If crow j is unaware of being followed, crow i moves toward this memory for exploitation; otherwise, crow i is relocated randomly for exploration. Thus, AP controls the probability of random exploration, whereas fl controls the movement step size. A larger fl encourages broader exploration, while a smaller fl supports local exploitation. CSA therefore balances exploration and exploitation through a simple memory-guided update with few control parameters and low computational overhead, making it suitable for repeated re-optimization in time-sensitive DTA scenarios.

3. Scenario Description and Problem Modeling

3.1. Scenario Description

We consider scenarios in which S UAVs $\mathcal{U} = \{U_1, U_2, \ldots, U_S\}$ observe Z targets $\mathcal{T} = \{T_1, T_2, \ldots, T_Z\}$ inside a three-dimensional (3D) mission space $[0, L_x] \times [0, L_y] \times [0, L_z] \subset \mathbb{R}^3$. The observation of targets is motivated by a wide range of practical UAV applications that require acquiring real-time information about ground objects or areas of interest, such as environmental monitoring, disaster damage assessment, and military reconnaissance. In these applications, each target carries an observational value that reflects the importance or urgency of obtaining information from it, and the collected value often decays over time or is degraded by sensor limitations and environmental factors. The main attributes of UAV U_i and target T_j are listed in Tables 1 and 2. To mathematically formulate the operational logic, we adopt the following assumptions:

- Each UAV and target is treated as a point mass, and the flight route is simplified to the Euclidean distance. While observing a target, the UAV is assumed to hover at the target location.
- A UAV may carry visible-light (VL), infrared (IR), and radar (RD) sensors. Environmental factors that degrade sensing performance are unified into sensor-specific effectiveness factors, which affect the overall capability through multi-sensor fusion.
- A centralized control center monitors the scenario, collects external intelligence, and updates the environmental state for dynamic task allocation.

The control center triggers dynamic task allocation immediately upon detecting any of the following four types of dynamic events:

- UAV events: sensor malfunctions or UAV crashes.
- Target events: new target appearances, target value decreases, or target movements.

- Environmental events: weather changes that degrade sensor performance.
- Preference events: shifts in decision-maker preferences.

Table 1. Attributes of UAV U_i.

Attribute	Symbol
Position at time t	$\boldsymbol{p}_i^t$
Flight speed	v_i
Remaining energy percentage at time t	E_i^t
Maximum endurance time	$t_{\text{endu},i}$
Vulnerability factor	ξ_i
Sensing capability at time t	$\eta_i(t)$
Sensor availability vector at time t	$\boldsymbol{I}_i^t$

Table 2. Attributes of target T_j.

Attribute	Symbol
Location at time t	$\boldsymbol{p}_j^t$
Observational value at time t	V_j^t
Required observation time	$t_{\text{obse},j}$
Completion status at time t	c_j^t

3.2. Task Allocation Problem Modeling

Let $\mathcal{A}^t$ denote the task-to-UAV allocation scheme at time t, which specifies the ordered task sequences of all UAVs. For UAV U_i, $\mathcal{A}_i^t = (T_{i,1}, T_{i,2}, \ldots, T_{i,|\mathcal{A}_i^t|})$ is the ordered list of targets assigned to it at time t. Let $\mathcal{T}_{\text{unfin}}^t = \{T_j \in \mathcal{T} \mid c_j^t = 0\}$ be the set of unfinished targets, where $c_j^t = 1$ indicates that target T_j has been completed and $c_j^t = 0$ otherwise. The DTA problem is formulated through the following objectives and constraints.

3.2.1. Objectives

The optimization problem is

$$\min_{\mathcal{A}^t} \boldsymbol{F}(\mathcal{A}^t, t) = \big(f_1(\mathcal{A}^t, t),\ f_2(\mathcal{A}^t, t)\big), \tag{3}$$

where f_1 is the remaining target value and f_2 is the mission makespan. The two objectives are selected to capture the main operational trade-off. The remaining target value reflects the mission reward, while the makespan reflects the mission cost in terms of time.

Remaining target value. The primary goal of multi-UAV observation missions is to maximize the total observational value collected from targets. The observational value of a target decays with time as information becomes outdated, and its acquisition through UAVs is degraded by sensor limitations and environmental factors. To formulate this as a minimization problem, we define the objective as the remaining unobserved target value:

$$f_1(\mathcal{A}^t, t) = \sum_{T_j \in \mathcal{T}_{\text{unfin}}^t} V_j^t \ - \sum_{i=1}^{S} \sum_{k \in \mathcal{A}_i^t} V_k^t \, \eta_i(t) \, p_{\text{surv},i}^{(k)} \, e^{-\lambda_{\text{obse}} \Delta t_{i,k}}, \tag{4}$$

where $e^{-\lambda_{\text{obse}} \Delta t_{i,k}}$ models the timeliness decay of target T_k's observational value with decay constant λ_{obse}, and $\Delta t_{i,k}$ is the time elapsed since UAV U_i reached target T_k. The sensing capability $\eta_i(t)$ follows a probabilistic multi-sensor fusion model:

$$\eta_i(t) = 1 - \prod_{m \in \mathcal{M}} \left(1 - I_{m,i}^t \cdot \omega_m^t\right), \tag{5}$$

where $\mathcal{M} \subseteq \{\mathrm{VL}, \mathrm{IR}, \mathrm{RD}\}$ is the set of sensors on UAV U_i, $I_{m,i}^t \in \{0,1\}$ is the availability status of sensor m, and $\omega_m^t \in (0,1]$ is the time-varying weather effectiveness factor for sensor m. This fusion model improves sensing capability and reliability in complex environments [28]. The weather factors ω_m^t are non-increasing during deterioration events, simulating worsening conditions such as dense fog or electromagnetic interference.

The survival probability $p_{\mathrm{surv},i}^{(k)}$ is derived from a cumulative path-dependent risk to provide an approximation. For a flight segment of length $d_{i,k}$ flown by UAV U_i toward target T_k according to its task sequence, the segment risk factor $\theta_{i,k}$ is defined as

$$\theta_{i,k} = 1 - e^{-\lambda_{\mathrm{risk}}\, \xi_i\, d_{i,k}}, \tag{6}$$

where λ_{risk} is a flight risk constant and ξ_i is the vulnerability factor of UAV U_i; it increases with payload weight, as heavier payloads enlarge the physical footprint and signatures. The cumulative survival probability is then updated recursively:

$$p_{\mathrm{surv},i}^{(k)} = p_{\mathrm{surv},i}^{(k-1)} \cdot \left(1 - \theta_{i,k}\right), \quad \text{with } p_{\mathrm{surv},i}^{(0)} = 1. \tag{7}$$

Mission makespan. In time-critical missions, an unbalanced workload can severely delay the overall response. To promote load balance and rapid mission completion, we minimize the makespan, i.e., the maximum individual completion time, across the swarm:

$$f_2(\mathcal{A}^t, t) = \max_{i \in \{1,\ldots,S\}} t_{\mathrm{comp},i}, \tag{8}$$

where $t_{\mathrm{comp},i} = t + \sum_{k \in \mathcal{A}_i^t} \left(d_{i,k}/v_i + t_{\mathrm{obse},k}\right)$ is the absolute completion time of UAV U_i after starting from its current position at time t, and $d_{i,k}$ denotes the Euclidean distance from its previous position to target T_k.

3.2.2. Constraints

Completeness constraint. To ensure no task is abandoned, every unfinished target at time t must be assigned to a UAV:

$$\bigcup_{i=1}^{S} \mathcal{A}_i^t = \mathcal{T}_{\mathrm{unfin}}^t. \tag{9}$$

Collaboration constraint. To avoid redundant assignments and ensure resource efficiency, each target is handled by exactly one UAV; the assigned target sets are pairwise disjoint:

$$\mathcal{A}_i^t \cap \mathcal{A}_j^t = \emptyset, \quad \forall i \neq j. \tag{10}$$

Energy constraint. To prevent mid-air energy depletion, each UAV must retain positive residual energy after completing its remaining tasks:

$$g_i(\mathcal{A}^t, t) = E_i^t - \gamma_i \left(t_{\mathrm{comp},i} - t\right) > 0, \tag{11}$$

where $\gamma_i = 1/t_{\mathrm{endu},i}$ is the energy consumption rate (percentage per unit time) of UAV U_i.

4. Multi-Swarm Dynamic Crow Search Algorithm for Task Allocation

To solve the DTA problem formulated in Section 3, we propose the multi-swarm dynamic crow search algorithm for task allocation (MDCSATA). Built upon a random-key encoding that maps discrete task assignments to a continuous search space compatible with CSA, the algorithm integrates five coordinated strategies to address the problem's dynamic, multi-objective, and constrained nature: (i) a violation-tolerant multi-swarm co-evolution balances feasibility and diversity; (ii) an objective-oriented heuristic initialization provides an informed start; (iii) an adaptive position update combines stagnation awareness, Lévy flights, and multi-source leaders; (iv) a stagnation and elite guided perturbation intensifies local search; and (v) an event-aware change response quantifies severity and adaptively regenerates solutions. Details of the encoding and each strategy are given in the following subsections.

Algorithm 1 first summarizes the overall workflow, after which the following subsections explain its main components.

Algorithm 1 Pseudo-code of MDCSATA

Input: Total swarm size N, objective count M, parameters AP, $fl_{\min}$, fl, ϵ_0
Output: Approximated PS in $\boldsymbol{A}_{\text{feas}}$

```
Initialize M sub-swarms with size of ⌊N/M⌋ via the heuristic initialization;
Evaluate all crows and initialize their memories with mt = 1;
Initialize A_feas and A_inf and perform the multi-swarm co-evolution;
while termination criterion not met do
    if dynamic events triggered then
        Execute the event-aware change response;
    end if
    for each sub-swarm do
        for each crow i in the sub-swarm do
            Select a leader j via the multi-source rule;
            Update the position of crow i via (15);
        end for
    end for
    Execute stagnation-triggered perturbation on crows with mt > 1;
    Evaluate new positions and update memories and mt counters;
    Update A_feas and A_inf using crows with mt = 1;
    Execute elite perturbation on A_feas;
    Perform the multi-swarm co-evolution;
end while
```

4.1. Random-Key Task Allocation Encoding

Task allocation is inherently combinatorial, yet CSA operates in a continuous search space. We adopt the random-key representation [29,30] to bridge this gap: the fractional parts of real-valued variables serve as sort keys for the discrete assignment, so any real-valued vector maps to a structurally valid allocation. By design, this construction satisfies the completeness and collaboration constraints in Section 3.2.2, eliminates repair operators, and makes the search space fully continuous, thereby fitting continuous optimizers such as CSA.

Each candidate allocation is represented as a variable-length random-key vector $\boldsymbol{x} = (x_1, x_2, \ldots, x_Z) \in [0, S)^Z$, where $Z = |\mathcal{T}^t|$ is the total number of targets appeared up to time t; Z grows monotonically as new targets appear. The j-th variable x_j encodes both the UAV assigned to target T_j and the target's execution priority within that UAV: the integer part $\lfloor x_j \rfloor \in \{0, 1, \ldots, S-1\}$ identifies the assigned UAV, and the fractional part $\{x_j\}$ gives the within-UAV execution order that smaller value denotes earlier execution.

Figure 1 illustrates the encoding and decoding process with six variables: five existing targets T_1–T_5 and one newly appeared target NT_1. Among them, completed target T_1 acts as a placeholder and is skipped during decoding; ongoing task T_5 is locked to U_2 with its fractional part set to zero, ensuring execution precedence; new target NT_1 is appended to the vector; and the remaining unstarted targets T_2, T_3, T_4 are free variables. The integer part of each variable identifies the assigned UAV, and the fractional part determines the within-UAV execution priority. The allocation scheme is obtained by discarding placeholders, grouping the remaining variables by assigned UAV, and sorting each group in ascending fractional order. The resulting allocation assigns (T_4, NT_1) to U_1 and (T_5, T_3, T_2) to U_2. This encoding ensures that the vector never shrinks, existing index mappings remain stable, the completeness and collaboration constraints are satisfied, and the continuity of in-progress observations is preserved without repair operators.

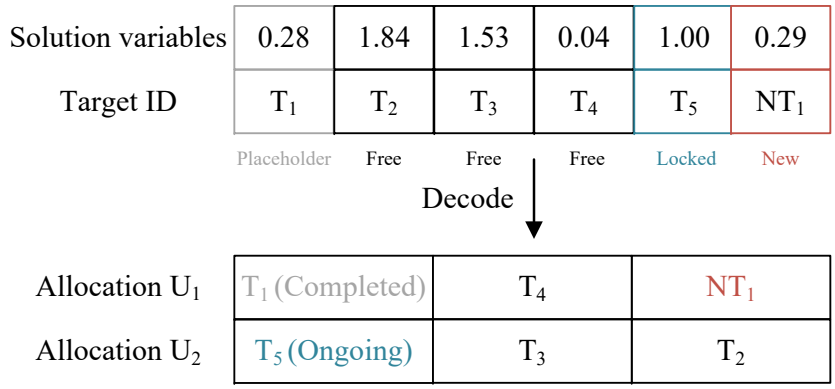


Figure 1. Random-key task allocation encoding illustration.

In Figure 1, the completed target colored in gray is skipped during decoding, the ongoing target colored in light blue is locked because it is already under execution, and the free targets colored in black together with the newly appeared target colored in red can be reassigned and reordered. The encoding ensures that each unfinished target is assigned exactly once; UAVs with no allocated target may be temporarily idle.

4.2. Violation-Tolerant Multi-Swarm Co-Evolution

MDCSATA adopts a multi-population-for-multi-objective paradigm, which maintains selection pressure along each Pareto direction while sharing information across populations [31–33]. Let N be the total swarm size; the algorithm maintains M sub-swarms of size $\lfloor N/M \rfloor$, each dedicated to one objective. In this study, $M = 2$, so the two sub-swarms are designated to f_1 and f_2, respectively. The term co-evolution refers to their interaction through swarm merging, shared feasible and infeasible archives, and objective-specific reallocation. In each iteration, solutions from all sub-swarms are merged and reallocated via a three-tier hierarchical sort that enforces relaxed feasibility:

1. Feasible solutions: solutions that satisfy all constraints;
2. Violation tolerated solutions: infeasible solutions whose constraint violation is within the tolerance threshold ϵ;
3. Strictly infeasible solutions: infeasible solutions whose constraint violation exceeds the tolerance threshold ϵ.

The feasible and violation-tolerated candidates are pooled and ranked independently for each sub-swarm. They are sorted by f_1 to refill the f_1-oriented sub-swarm and by f_2 to refill the f_2-oriented sub-swarm, with the best $\lfloor N/M \rfloor$ candidates retained in each case. For example, between two candidates, the one with the lower f_1 is ranked first for the f_1-oriented sub-swarm, whereas the one with the lower f_2 is ranked first for the f_2-oriented

sub-swarm. If the first two tiers together yield fewer than $\lfloor N/M \rfloor$ solutions, the remaining slots are filled with strictly infeasible solutions from the third tier. The tolerance threshold decays with iteration progress to encourage early exploration and late strict feasibility:

$$\epsilon(\tau) = \epsilon_0 \cdot \left(1 - \tau/\tau_t\right)^2 \tag{12}$$

where $\epsilon_0 = |\mathcal{U}|$ is the initial tolerance, τ is the current iteration, and τ_t is the maximum iteration count at the current time t.

Instead of a single archive, MDCSATA uses a dual-archive structure: a feasible archive $\boldsymbol{A}_{\text{feas}}$ and an infeasible archive $\boldsymbol{A}_{\text{inf}}$, maintained by crowding-distance truncation [34]. $\boldsymbol{A}_{\text{feas}}$ stores non-dominated feasible solutions and guides the main search, while $\boldsymbol{A}_{\text{inf}}$ stores infeasible non-dominated solutions. During leader selection in Section 4.4, infeasible individuals have a 50% chance of following a leader from $\boldsymbol{A}_{\text{inf}}$, which helps explore along the constraint boundary to discover feasible regions.

Based on the above, the memory update adopts a violation-tolerant rule that replaces standard Pareto dominance. For a crow i, let mt_i be its stagnation counter, which records the number of consecutive iterations without memory improvement. The update proceeds as follows:

- If $\boldsymbol{x}$ ϵ-dominates $\boldsymbol{m}$, the memory is replaced with $\boldsymbol{x}$ and mt_i is reset to 1.
- If $\boldsymbol{x}$ and $\boldsymbol{m}$ are ϵ-equivalent, the designated objective of the crow's corresponding sub-swarm acts as a tiebreaker: if $\boldsymbol{x}$ wins, it replaces the memory, and mt_i is reset to 1. Regardless of the tiebreaker, $\boldsymbol{x}$ is always added to the appropriate archive—$\boldsymbol{A}_{\text{feas}}$ if feasible, or $\boldsymbol{A}_{\text{inf}}$ otherwise.
- Otherwise, $\boldsymbol{m}$ ϵ-dominates $\boldsymbol{x}$, the memory is retained, and mt_i is increased by 1.

4.3. Objective-Oriented Heuristic Initialization

To accelerate convergence toward the PF and preserve diversity across the objective space, a composite initialization strategy is designed. Each sub-swarm is initialized by combining one space-filling sampler with two problem-aware heuristics, yielding a more informed start than random initialization.

Fifty percent of the individuals are generated via Latin hypercube sampling (LHS). Unlike uniform random sampling, LHS stratifies each dimension into equal-probability intervals and draws exactly one sample per interval, reducing clustering and improving decision-space coverage for a given sample size [35]. This offers a stronger foundation for global exploration.

Another 25% are generated by a spatiotemporal greedy heuristic: each target is assigned to its currently nearest UAV, and the within-UAV task order is sorted in descending order of the value-to-distance ratio $V_k^t / \max(d_{i,k}, 1)$. The nearest-neighbor assignment shortens flight paths, reducing energy consumption and path-dependent risk. It also prioritizes high-value targets that can be reached quickly, mitigating the temporal decay of observational value in (4) and steering the swarms toward lower f_1.

The remaining 25% are constructed by a load-balanced heuristic that distributes unfinished targets across available UAVs in a round-robin fashion. Since the makespan f_2 in (8) is determined by the most heavily loaded UAV, balancing workload directly reduces f_2 and provides guiding solutions near its lower bound.

4.4. Adaptive Position Update

As discussed in Section 2, AP and fl are important CSA parameters that control exploitation and exploration. Accordingly, several studies have adaptively adjusted these parameters and reported improved performance [36]. Inspired by this, MDCSATA adapts the awareness probability AP and the flight length fl, updating the crows' positions as follows.

Multi-source leader selection. For each crow, a leader is drawn from one of three sources: $3/8$ probability by objective-based roulette-wheel selection from the relevant archive, which favors better fitness on the sub-swarm's designated objective; $3/8$ probability by binary tournament selection on crowding distance from the relevant archive, which favors diversity; and $1/4$ probability by random selection within its own sub-swarm to enhance diversity through non-elite solutions. These probabilities sum to one. The equal archive-based probabilities balance objective-oriented convergence and crowding-distance-based diversity, while the smaller intra-swarm probability preserves exploration through non-elite information without weakening the guidance of archived solutions. The relevant archive, either $\boldsymbol{A}_{\text{feas}}$ or $\boldsymbol{A}_{\text{inf}}$, is determined by the feasibility rule in Section 4.2. This composition provides both selection pressure and diversity preservation.

Stagnation-aware awareness probability. The adaptive awareness probability is

$$AP_i = AP \cdot \big(0.5 + \ln(1 + mt_i)\big), \tag{13}$$

so that crows trapped in local optima gradually increase their random search probability, facilitating a self-regulated recovery from stagnation.

Lévy flight length. When the following behavior is triggered, then the flight length is sampled from a Lévy distribution via Mantegna's algorithm with $\beta = 1.5$, then modulated by a cosine envelope to gradually transition the search from exploration to exploitation:

$$fl^{\tau} = fl_{\min} + \min\Big(\cos\big(\tfrac{\tau \bmod \tau_t}{\tau_t} \cdot \tfrac{\pi}{2}\big) \cdot |L(\beta)|,\ fl\Big) \tag{14}$$

where $fl_{\min}$ is the base flight length, fl is the flight length parameter used to truncate the Lévy-modulated increment, and $L(\beta)$ is a Lévy flight variable. Compared to Gaussian or uniform random walks, the heavy-tailed Lévy step produces frequent short moves with occasional long jumps. This foraging-like pattern is widely used in EAs to explore the search space more thoroughly and escape local optima [35]. The cosine envelope gradually reduces the size of long jumps as τ approaches τ_t, so the search smoothly transitions from broad exploration to exploitation.

The selected leader affects crow i through its memory, which corresponds to the food-hiding position followed in CSA. Combining the above components, the position update formula is

$$\boldsymbol{x}_i^{\tau+1} = \begin{cases} \boldsymbol{x}_i^{\tau} + r_i \cdot fl^{\tau} \cdot \big(\boldsymbol{m}_j^{\tau} - \boldsymbol{x}_i^{\tau}\big), & r_j \geq AP_i \\ \text{a random position}, & r_j < AP_i \end{cases} \tag{15}$$

Moreover, the variables exceeding the search bounds are clamped to the boundary after the position update.

4.5. Stagnation and Elite Guided Perturbation

To intensify local exploitation, MDCSATA incorporates two perturbation operators that target stagnated and elite solutions, respectively.

Stagnation-triggered perturbation. For each crow with $mt > 1$, two variables that are not part of any ongoing task are randomly selected, and their integer parts are swapped while the fractional parts are kept unchanged. The perturbed solution is immediately re-

evaluated. If the new solution ϵ-dominates the crow's memory, the memory is replaced and mt is reset to 1. If the two are non-dominated, the new solution is added to the appropriate archive. Otherwise, the variable changes are rolled back to prevent degradation. This mechanism provides targeted repair for stagnated solutions without undirected disruption.

Elite perturbation. At the end of each iteration, $\lceil N/(5M) \rceil$ solutions are randomly selected from the feasible archive $\boldsymbol{A}_{\text{feas}}$ for elite perturbation [31]. For each selected elite, we apply a Gaussian perturbation with standard deviation $0.1 \cdot (x_j^{\max} - x_j^{\min})$ to a randomly chosen free variable x_j. Any mutant that is not worse than the original under ϵ-dominance is added to the archive. Since most perturbations only alter the fractional part, this operator primarily fine-tunes the execution order within each UAV, thereby helping to search along the PF.

4.6. Event-Aware Change Response

In DTA problems, dynamic events induce abrupt changes in the problem landscape. A complete restart would discard the evolutionary information from previous problem environments, whereas a simple warm start may fail to adapt to severe changes. MDC-SATA therefore employs an event-aware severity quantification and an adaptive regeneration strategy.

After an environmental change, each solution is first dimension-adapted: newly appeared targets are appended to the vector, completed targets are retained only as skipped placeholders, ongoing targets are locked to their executing UAVs, and assignments to crashed UAVs are released for reallocation. Then $K = \min(10, |\boldsymbol{A}_{\text{feas}}|)$ elite solutions are randomly selected from $\boldsymbol{A}_{\text{feas}}$ and re-evaluated. The severity δ is defined as

$$\delta = \delta_{\text{base}} + \frac{1}{K \cdot M} \sum_{i=1}^{K} \sum_{\ell=1}^{M} \frac{\left| f_\ell(\boldsymbol{x}_i)' - f_\ell(\boldsymbol{x}_i) \right|}{|f_\ell(\boldsymbol{x}_i)|} \tag{16}$$

where $f_\ell(\boldsymbol{x}_i)$ and $f_\ell(\boldsymbol{x}_i)'$ are the pre-change and post-change objective values, and $\delta_{\text{base}} = 0.15$ is added when the event set includes new target appearances or UAV crashes, which may not be fully captured by objective changes from re-evaluation alone. This base term represents a minimal structural-change penalty, while the actual extent of the change is still reflected by the re-evaluation term.

Based on δ, each sub-swarm's new population is composed of three adaptive components:

1. Elite retention ($R_{\text{keep}} = \max(0.2,\ 0.6 - \min(3\delta, 0.4))$): the best R_{keep} fraction of the union of the sub-swarm and the re-evaluated elites is retained for maintaining solution diversity.
2. Heuristic injection ($R_{\text{heur}} = \min(0.5,\ 0.2 + \min(3\delta, 0.3))$): balanced and greedy heuristics are applied to generate solutions that help guide the search after dynamic events.
3. Global exploration ($R_{\text{lhs}} = 1 - R_{\text{keep}} - R_{\text{heur}}$): LHS solutions are generated to distribute in the reshaped search space.

These ratios retain historical information when the change severity is small and increase guided reconstruction and global exploration as the estimated severity grows. Ongoing-task variables are locked within the heuristic and LHS components to preserve the continuity of in-progress observations. After that, both archives are re-initialized and the multi-swarm co-evolution is performed.

4.7. Overall Procedure and Time Complexity

The main optimization steps are summarized in Algorithm 1. Here is a simple example of the process: consider two UAVs and three targets. A crow at $\boldsymbol{x} = (0.2, 1.6, 1.3)$ is decoded as $U_1 : (T_1)$ and $U_2 : (T_3, T_2)$ and then evaluated using the two objectives. Suppose its selected leader has memory $\boldsymbol{m} = (1.1, 0.4, 1.8)$ and $r_i fl^\tau = 0.5$. The following behavior produces $\boldsymbol{x}' = (0.65, 1.0, 1.55)$, which is decoded and evaluated as $U_1 : (T_1)$ and $U_2 : (T_2, T_3)$. If a new target subsequently appears, its variable is appended, while completed targets are skipped and ongoing targets remain locked before regeneration.

Now, we analyze the time complexity of MDCSATA with M objectives, swarm size N, problem dimension n, maximum iteration Γ, dynamic event count Θ, and objective evaluation cost F. Both archives are bounded by N. The complexities of the main stages are:

- Initialization: heuristic sorting costs $O(N \cdot n \log n)$ and the evaluation costs $O(N \cdot F)$.
- Position update: $O(\Gamma \cdot N \cdot n)$.
- Evaluation: $O(\Gamma \cdot N \cdot F)$.
- Multi-swarm co-evolution with violation-tolerant sorting: $O(\Gamma \cdot MN \log N)$.
- Dual-archive maintenance via crowding-distance truncation: $O(\Gamma \cdot N^2)$.
- Stagnation-triggered perturbation and elite perturbation: $O(\Gamma \cdot N(n + F))$.
- Event-aware change response: re-evaluation and regeneration cost $O(\Theta \cdot N(n \log n + F))$.

The overall complexity is $O\big((\Gamma + \Theta) \cdot N \cdot F + \Gamma \cdot N^2 + (\Gamma + \Theta) \cdot N \cdot n \log n\big)$. The objective evaluation is dominated by the survival-probability integration along flight segments, giving $F \gg n \log n$. Consequently, the effective complexity simplifies to $O\big((\Gamma + \Theta) \cdot N \cdot F\big)$, indicating that the auxiliary mechanisms impose limited overhead compared with the indispensable objective evaluations.

5. Experiments and Results

5.1. Experimental Setup

5.1.1. Scenarios Settings

Three 3D scenarios are constructed to evaluate MDCSATA comprehensively: S_1, S_2, and S_3. All share an 8000 m × 8000 m × 200 m operational airspace. Following a top-down hierarchical derivation, the states of S_2 and S_1 are extracted from S_3, ensuring spatial consistency and physical inheritance across scenarios. λ_{obse} and λ_{risk} are set to 0.001 and 0.0002, respectively, to align with the scale of the operational airspace.

Scenarios S_1, S_2, and S_3 involve 3, 4, and 5 UAVs, and 10, 14, and 18 initial targets, respectively. There are two types of UAVs in the scenarios, and detailed attributes are listed in Table 3. Target configurations are provided in Table 4. A sequence of dynamic events is injected across the scenarios, following a subset hierarchy: S_1 experiences fewer and sparser events than S_2, which in turn has fewer events than S_3, while all share the same trigger times as S_3. All events are detailed in Table 5.

Table 3. Attribute of UAVs across the scenarios.

ID	Type	Speed (m/s)	Endurance (s)	VL	IR	Radar	Vulnerability	Position (x, y, z)	Involved Scenarios
U_1	A	15	1800	1	1	0	0.9	(1466, 73, 80)	$S_1/S_2/S_3$
U_2	B	15	2400	1	1	1	1.0	(1373, 545, 76)	$S_1/S_2/S_3$
U_3	A	15	1800	1	1	0	0.9	(992, 1091, 73)	$S_1/S_2/S_3$
U_4	B	15	2400	1	1	1	1.0	(557, 1390, 72)	S_2/S_3
U_5	A	15	1800	1	1	0	0.9	(70, 1400, 75)	S_3

Table 4. Attribute of targets across the scenarios.

ID	Observational Value	Required Time (s)	Position (x, y, z)	Involved Scenarios
T_1	67	12	(1786, 3385, 82)	$S_1/S_2/S_3$
T_2	52	11	(5761, 1668, 105)	$S_1/S_2/S_3$
T_3	84	16	(910, 5741, 78)	$S_1/S_2/S_3$
T_4	78	14	(5359, 7562, 78)	$S_1/S_2/S_3$
T_5	92	17	(1948, 1934, 82)	$S_1/S_2/S_3$
T_6	56	11	(4517, 2712, 90)	$S_1/S_2/S_3$
T_7	57	11	(3873, 5182, 89)	$S_1/S_2/S_3$
T_8	59	10	(4886, 6545, 140)	$S_1/S_2/S_3$
T_9	91	18	(3267, 1243, 86)	$S_1/S_2/S_3$
T_{10}	99	19	(7238, 2633, 82)	$S_1/S_2/S_3$
T_{11}	97	19	(2549, 4163, 88)	S_2/S_3
T_{12}	71	14	(5627, 5859, 134)	S_2/S_3
T_{13}	89	17	(2874, 3164, 80)	S_2/S_3
T_{14}	90	17	(4406, 3981, 81)	S_2/S_3
T_{15}	71	13	(2025, 5425, 88)	S_3
T_{16}	98	19	(7516, 6138, 111)	S_3
T_{17}	99	20	(1533, 1018, 77)	S_3
T_{18}	68	14	(5192, 233, 82)	S_3
NT_1	76	15	(2184, 6889, 83)	$S_1/S_2/S_3$
NT_2	60	11	(6792, 7246, 80)	S_3

Table 5. Dynamic events across the scenarios.

No.	Time (s)	Description	Involved Scenarios
1	100	UAV U_5 crashed	S_3
2	100	New target NT_1 appears at (2184, 6889, 83)	$S_1/S_2/S_3$
3	170	Decision preference shifted from $[0.5, 0.5]$ to $[0.75, 0.25]$	S_3
4	170	Target T_{12} value decreased from 71 to 35	S_2/S_3
5	250	UAV U_2 sensor VL damaged	S_3
6	250	UAV U_3 crashed	$S_1/S_2/S_3$
7	330	Weather deteriorated sensor effectiveness factor (VL: 0.94, IR: 0.58, RD: 0.54)	S_3
8	330	UAV U_1 sensor IR damaged	S_2/S_3
9	440	Target T_5 moved from (1948, 1934, 82) to (1537, 1365, 125)	S_3
10	440	Decision preference shifted from $[0.5, 0.5]$ to $[0.75, 0.25]$	$S_1/S_2/S_3$
11	540	Target T_{13} value decreased from 89 to 44	S_3
12	540	Target NT_1 moved from (2184, 6889, 83) to (1206, 6405, 129)	S_2
13	540	Target T_{15} moved from (2025, 5425, 88) to (1206, 6405, 129)	S_3
14	660	New target NT_2 appears at (6792, 7246, 80)	S_3
15	660	Weather deteriorated sensor effectiveness factor (VL: 0.75, IR: 0.76, RD: 0.91)	$S_1/S_2/S_3$

5.1.2. Comparison Algorithms

MDCSATA is compared with seven DMOEAs. In addition to DMOAWPSO, DCTAEA, and DCMPSO introduced in Section 1, four classical DMOEAs are employed:

- Dynamic non-dominated sorting genetic algorithm II (DNSGA-II) [37]: extends NSGA-II with a dynamic response that replaces a fraction of the population by randomly generated or mutated individuals upon detecting a change.
- Steady-state and generational evolutionary algorithm (SGEA) [38]: combines steady-state and generational updates; upon a change, half of the population estimates the PS movement trend, and the remainder is reinitialized using the predicted trend.
- Multi-objective evolutionary algorithm based on decomposition with support vector regression (MOEA/D-SVR) [39]: trains support vector regression models on historical

solution data to predict the decision variables of new solutions when a problem change is detected.

- Distributed bi-behaviors crow search algorithm II (DBCSA-II) [27]: a CSA variant for DMOPs that applies Gaussian-beta functions to enhance search performance, retaining high-quality elites and randomly re-initializing low-quality solutions upon change.

5.1.3. Parameter Settings

All algorithms share the population size $N = 100$, the archive size of 100, and the same total number of iterations per scenario. Each run begins with an initial optimization phase of $2\tau_t$ iterations; thereafter, a re-optimization phase of τ_t iterations is triggered upon each dynamic event, with $\tau_t = 20$. At the end of each phase, the solution to be executed is selected from the current PS according to the decision-maker's preference weights, which are initially set to $[0.5, 0.5]$ for equal priority to value and makespan. Each algorithm is independently executed for 30 runs. Parameters specific to each comparison algorithm follow their original papers, except for population size, archive size, and iteration numbers.

The specific parameters of MDCSATA are listed in Table 6.

Table 6. Parameter settings of MDCSATA.

Parameter	Symbol	Value
Total swarm size	N	100
Sub-swarm size	$\lfloor N/M \rfloor$	50
Awareness probability	AP	0.05
Flight length	fl	1.5
Minimum flight length	$fl_{\min}$	1.0

5.1.4. Performance Metrics

Two standard DMOP metrics, mean hypervolume (MHV) and mean inverted generational distance (MIGD), are adopted [21]. Let $\mathrm{Y}_t = \{1, 2, \ldots, |\mathrm{Y}_t|\}$ denote the ordered sequence of environmental time steps in a single run, $\hat{PF}_t$ the PF approximation obtained at time t, and PF_t^* the corresponding reference PF constructed from the classic DNSGA-II with large population and iteration budgets at that step. MHV and MIGD are defined as:

The hypervolume $\mathrm{HV}(\hat{PF}_t, \boldsymbol{r}_t)$ measures the volume of the objective space dominated by $\hat{PF}_t$ relative to a reference point $\boldsymbol{r}_t$. MHV averages this quantity across all time steps:

$$\mathrm{MHV} = \frac{1}{|\mathrm{Y}_t|} \sum_{t \in \mathrm{Y}_t} \mathrm{HV}(\hat{PF}_t, \boldsymbol{r}_t) \tag{17}$$

The per-step reference point $\boldsymbol{r}_t$ is set to 1.1 times the per-objective maximum over the union of PF_t^* and all algorithms' PFs at time t. A higher MHV indicates better performance.

IGD at time t measures the average distance from each point on PF_t^* to its nearest point in $\hat{PF}_t$, jointly reflecting convergence and diversity. MIGD averages this across all time steps:

$$\mathrm{MIGD} = \frac{1}{|\mathrm{Y}_t|} \sum_{t \in \mathrm{Y}_t} \frac{1}{|PF_t^*|} \sum_{\boldsymbol{v} \in PF_t^*} \min_{\boldsymbol{u} \in \hat{PF}_t} \|\boldsymbol{v} - \boldsymbol{u}\| \tag{18}$$

A lower MIGD indicates better tracking of the moving PF.

5.2. Results and Analysis

5.2.1. Performance Comparison

Tables 7 and 8 report the mean ± std of MHV and MIGD over 30 runs of all algorithms across the three scenarios. Statistical significance is assessed by the Wilcoxon rank-sum test, and the p-value results are presented in Table 9.

MDCSATA achieves the best MHV and MIGD on all three scenarios, with relatively low standard deviations that indicate stable performance across independent runs. In S_1, all comparisons are highly significant after Bonferroni correction, with MDCSATA outperforming DCTAEA and DNSGA-II by 7.69% and 8.49% in MHV, and by 32.00% and 39.92% in MIGD, respectively. In S_2, DCTAEA is the only baseline whose MHV does not significantly differ from MDCSATA after Bonferroni correction, although MDCSATA still achieves a 0.93% improvement; DNSGA-II remains significantly worse in MHV. In MIGD, MDCSATA outperforms DCTAEA by 12.92% and DNSGA-II by 20.05%, demonstrating better preservation of front diversity. In S_3, MDCSATA's MHV advantage over DCTAEA is not statistically significant, but the 17.89% MIGD improvement over DCTAEA is significant. The advantage over DNSGA-II in S_3 is significant for MHV at the 0.05 level and highly significant for MIGD. Across all settings, the only comparisons that fail to reach statistical significance are the MHV comparisons against DCTAEA in S_2 and S_3; every other pairwise test confirms the superiority of MDCSATA.

Table 7. MHV results across different scenarios.

Algorithm	S_1	S_2	S_3
MDCSATA	$\mathbf{2.279 \times 10^5} \pm 1.626 \times 10^4$	$\mathbf{4.453 \times 10^5} \pm 2.491 \times 10^4$	$\mathbf{5.759 \times 10^5} \pm 4.453 \times 10^4$
DMOAWPSO	$2.059 \times 10^5 \pm 1.920 \times 10^4$	$3.855 \times 10^5 \pm \mathbf{2.442 \times 10^4}$	$4.603 \times 10^5 \pm 4.326 \times 10^4$
DCTAEA	$2.116 \times 10^5 \pm \mathbf{8.890 \times 10^3}$	$4.411 \times 10^5 \pm 3.121 \times 10^4$	$5.510 \times 10^5 \pm 4.513 \times 10^4$
DCMPSO	$1.890 \times 10^5 \pm 1.873 \times 10^4$	$3.213 \times 10^5 \pm 3.331 \times 10^4$	$3.838 \times 10^5 \pm 4.582 \times 10^4$
DNSGA-II	$2.101 \times 10^5 \pm 2.229 \times 10^4$	$4.211 \times 10^5 \pm 2.509 \times 10^4$	$5.406 \times 10^5 \pm 4.237 \times 10^4$
SGEA	$2.024 \times 10^5 \pm 1.398 \times 10^4$	$4.038 \times 10^5 \pm 3.595 \times 10^4$	$5.130 \times 10^5 \pm 4.908 \times 10^4$
MOEADSVR	$1.838 \times 10^5 \pm 2.365 \times 10^4$	$3.638 \times 10^5 \pm 4.085 \times 10^4$	$4.536 \times 10^5 \pm 5.036 \times 10^4$
DBCSA-II	$1.333 \times 10^5 \pm 2.195 \times 10^4$	$2.135 \times 10^5 \pm 3.661 \times 10^4$	$2.352 \times 10^5 \pm \mathbf{3.631 \times 10^4}$

Note: Bold denotes the best result.

To further quantify the advantage, the per-algorithm improvement ratios on MHV and MIGD across the three scenarios are calculated. MDCSATA outperforms every baseline on both metrics in all scenarios. The largest gains are observed against DBCSA-II, where MHV improves by 71.00% in S_1, 108.54% in S_2, and 144.84% in S_3, while MIGD improves by 74.44%, 80.58%, and 79.14%, respectively. Against DCMPSO, MHV gains range from 20.55% in S_1 to 50.06% in S_3, and MIGD gains range from 51.93% to 64.08%. Even against the strongest competitors, MDCSATA maintains consistent advantages: MHV improvements over DCTAEA and DNSGA-II reach 0.93%–8.49%, and MIGD improvements reach 12.92–39.92%, with only two comparisons lacking statistical significance. These consistent margins across metrics and scenarios underscore the superiority of MDCSATA.

Table 8. MIGD results across different scenarios.

Algorithm	S_1	S_2	S_3
MDCSATA	$\mathbf{8.010 \times 10^1} \pm \mathbf{2.285 \times 10^1}$	$\mathbf{8.342 \times 10^1} \pm \mathbf{1.521 \times 10^1}$	$\mathbf{1.297 \times 10^2} \pm \mathbf{2.699 \times 10^1}$
DMOAWPSO	$1.250 \times 10^2 \pm 4.043 \times 10^1$	$1.524 \times 10^2 \pm 2.829 \times 10^1$	$2.502 \times 10^2 \pm 4.628 \times 10^1$
DCTAEA	$1.178 \times 10^2 \pm 2.515 \times 10^1$	$9.580 \times 10^1 \pm 2.226 \times 10^1$	$1.579 \times 10^2 \pm 3.814 \times 10^1$
DCMPSO	$1.666 \times 10^2 \pm 3.715 \times 10^1$	$2.273 \times 10^2 \pm 4.228 \times 10^1$	$3.610 \times 10^2 \pm 5.793 \times 10^1$
DNSGA-II	$1.333 \times 10^2 \pm 3.668 \times 10^1$	$1.043 \times 10^2 \pm 2.068 \times 10^1$	$1.674 \times 10^2 \pm 4.802 \times 10^1$
SGEA	$1.429 \times 10^2 \pm 3.736 \times 10^1$	$1.432 \times 10^2 \pm 4.565 \times 10^1$	$2.042 \times 10^2 \pm 5.299 \times 10^1$
MOEADSVR	$2.022 \times 10^2 \pm 5.846 \times 10^1$	$2.018 \times 10^2 \pm 5.626 \times 10^1$	$2.963 \times 10^2 \pm 6.919 \times 10^1$
DBCSA-II	$3.133 \times 10^2 \pm 6.046 \times 10^1$	$4.296 \times 10^2 \pm 6.473 \times 10^1$	$6.218 \times 10^2 \pm 6.328 \times 10^1$

Note: Bold denotes the best result.

Table 9. The p-value results of comparative Wilcoxon rank-sum test.

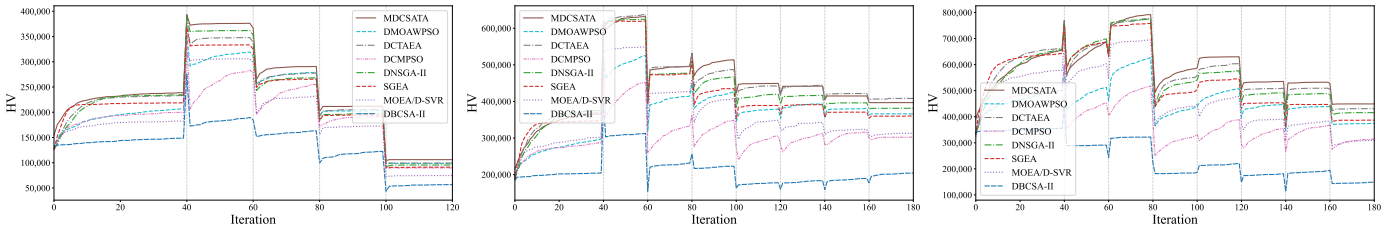
Scenario	Metric	DMOAWPSO	DCTAEA	DCMPSO	DNSGA-II	SGEA	MOEA/D-SVR	DBCSA-II
S_1	MHV	1.868×10^{-05} **	2.959×10^{-05} **	3.474×10^{-10} **	1.236×10^{-03} **	1.596×10^{-07} **	1.547×10^{-09} **	3.020×10^{-11} **
	MIGD	1.385×10^{-06} **	5.600×10^{-07} **	7.389×10^{-11} **	1.359×10^{-07} **	1.698×10^{-08} **	5.494×10^{-11} **	3.020×10^{-11} **
S_2	MHV	1.695×10^{-09} **	4.643×10^{-01}	3.020×10^{-11} **	4.218×10^{-04} **	3.592×10^{-05} **	3.825×10^{-09} **	3.020×10^{-11} **
	MIGD	4.504×10^{-11} **	2.068×10^{-02} *	3.020×10^{-11} **	3.006×10^{-04} **	1.254×10^{-07} **	8.993×10^{-11} **	3.020×10^{-11} **
S_3	MHV	1.206×10^{-10} **	6.353×10^{-02}	3.020×10^{-11} **	1.123×10^{-02} *	1.996×10^{-05} **	6.722×10^{-10} **	3.020×10^{-11} **
	MIGD	6.696×10^{-11} **	5.322×10^{-03} **	3.020×10^{-11} **	6.203×10^{-04} **	3.256×10^{-07} **	6.696×10^{-11} **	3.020×10^{-11} **

Note: * and ** denotes the p-value is lower than the significance level 0.05 and 0.05/7 for Bonferroni correction, respectively.

Among the baselines, DBCSA-II and DCMPSO exhibit the worst performance. DMOAWPSO, SGEA, and MOEA/D-SVR all rank consistently below MDCSATA, with significant gaps on both metrics. These results collectively validate that the tailored strategies of MDCSATA enable efficient task allocation across diverse dynamic scenarios.

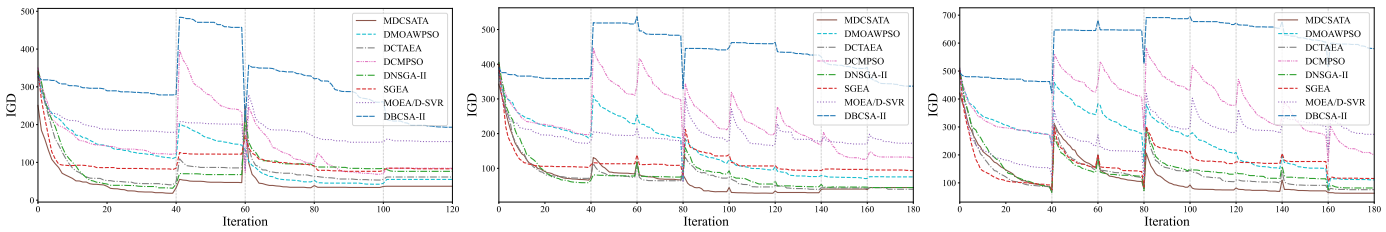
Figures 2 and 3 show the per-iteration HV and IGD convergence curves for all three scenarios. MDCSATA leads all algorithms from the earliest iterations, confirming that the objective-oriented heuristic initialization provides a well-informed starting population. It also converges faster within each environmental time step, reflecting the combined effect of the adaptive position update and the stagnation and elite guided perturbation. After each dynamic event, MDCSATA's metric value recovers rapidly. This resilience is driven by the event-aware change response: the severity metric δ quantifies each event's impact and adjusts the regeneration ratios accordingly. As a result, MDCSATA reaches a higher performance level within each problem environment faster than the other algorithms.

By contrast, DNSGA-II uses a fixed-ratio random injection, which fails to redirect the population efficiently after disruptions such as UAV crashes, leading to slower performance recovery. SGEA and MOEA/D-SVR rely on prediction-based reinitialization, which becomes inaccurate under compound events and incurs a high change-response cost. DBCSA-II and DCMPSO exhibit curves that differ from others, consistent with the results reported above.



(**a**) HV iteration curves of S_1. (**b**) HV iteration curves of S_2. (**c**) HV iteration curves of S_3.

Figure 2. HV iteration curves of the algorithms across different scenarios.



(**a**) IGD iteration curves of S_1. (**b**) IGD iteration curves of S_2. (**c**) IGD iteration curves of S_3.

Figure 3. IGD iteration curves of the algorithms across different scenarios.

5.2.2. Ablation Study

To examine the contribution of each proposed strategy, an incremental ablation study is conducted with all settings identical to those described in Section 5.1. The baseline MOCSA is a dynamic MOCSA core that includes a single-swarm CSA update, Pareto archive, and restart-based dynamic response. Starting from MOCSA, the proposed strategies are added sequentially. $MOCSA_M$ introduces the violation-tolerant multi-swarm co-evolution mechanism; $MOCSA_{MH}$ further adds objective-oriented heuristic initialization; $MOCSA_{MHA}$ adds the adaptive position update strategy; $MOCSA_{MHAP}$ adds stagnation and elite guided perturbation; and MDCSATA further activates the event-aware change response. The MHV and MIGD results are reported in Tables 10 and 11.

Table 10. MHV results of the ablation study.

Variant	S_1	S_2	S_3
MOCSA	$1.838 \times 10^5 \pm 2.014 \times 10^4$	$2.861 \times 10^5 \pm 3.006 \times 10^4$	$3.283 \times 10^5 \pm \mathbf{2.801 \times 10^4}$
$MOCSA_M$	$1.982 \times 10^5 \pm 2.018 \times 10^4$	$3.415 \times 10^5 \pm 3.359 \times 10^4$	$3.565 \times 10^5 \pm 4.527 \times 10^4$
$MOCSA_{MH}$	$2.267 \times 10^5 \pm 1.732 \times 10^4$	$4.097 \times 10^5 \pm 3.342 \times 10^4$	$5.052 \times 10^5 \pm 3.053 \times 10^4$
$MOCSA_{MHA}$	$2.182 \times 10^5 \pm \mathbf{1.524 \times 10^4}$	$4.270 \times 10^5 \pm 3.326 \times 10^4$	$5.536 \times 10^5 \pm 4.285 \times 10^4$
$MOCSA_{MHAP}$	$2.274 \times 10^5 \pm 1.591 \times 10^4$	$\mathbf{4.461 \times 10^5} \pm 3.371 \times 10^4$	$5.659 \times 10^5 \pm 4.691 \times 10^4$
MDCSATA	$\mathbf{2.279 \times 10^5} \pm 1.626 \times 10^4$	$4.453 \times 10^5 \pm \mathbf{2.491 \times 10^4}$	$\mathbf{5.759 \times 10^5} \pm 4.453 \times 10^4$

Note: Bold denotes the best result.

The complete MDCSATA consistently improves over the dynamic MOCSA core by a large margin. Compared with MOCSA, MDCSATA increases MHV by 24.0%, 55.6%, and 75.4% in S_1, S_2, and S_3, respectively, while reducing MIGD by 55.8%, 70.7%, and 71.0%. These results confirm that the performance advantage of MDCSATA does not come merely from the problem-specific encoding, but from the proposed algorithmic strategies.

Table 11. MIGD results of the ablation study.

Variant	S_1	S_2	S_3
MOCSA	$1.811 \times 10^2 \pm 4.711 \times 10^1$	$2.843 \times 10^2 \pm 4.948 \times 10^1$	$4.473 \times 10^2 \pm 4.125 \times 10^1$
$MOCSA_M$	$1.571 \times 10^2 \pm 4.044 \times 10^1$	$2.061 \times 10^2 \pm 4.345 \times 10^1$	$3.853 \times 10^2 \pm 5.465 \times 10^1$
$MOCSA_{MH}$	$9.382 \times 10^1 \pm 3.128 \times 10^1$	$1.114 \times 10^2 \pm 2.203 \times 10^1$	$1.854 \times 10^2 \pm 2.736 \times 10^1$
$MOCSA_{MHA}$	$9.882 \times 10^1 \pm 2.395 \times 10^1$	$9.929 \times 10^1 \pm 1.876 \times 10^1$	$1.482 \times 10^2 \pm 2.968 \times 10^1$
$MOCSA_{MHAP}$	$8.174 \times 10^1 \pm \mathbf{1.978 \times 10^1}$	$8.946 \times 10^1 \pm 1.535 \times 10^1$	$1.316 \times 10^2 \pm 2.799 \times 10^1$
MDCSATA	$\mathbf{8.010 \times 10^1} \pm 2.285 \times 10^1$	$\mathbf{8.342 \times 10^1 \pm 1.521 \times 10^1}$	$\mathbf{1.297 \times 10^2 \pm 2.699 \times 10^1}$

Note: Bold denotes the best result.

The first increment from MOCSA to $MOCSA_M$ yields improvements in all scenarios, indicating that the violation-tolerant multi-swarm co-evolution mechanism helps preserve feasible search directions and front diversity under dynamic constraints. The most pronounced gain is observed from $MOCSA_M$ to $MOCSA_{MH}$. After objective-oriented heuristic initialization is introduced, MHV increases by 14.4%, 20.0%, and 41.7%, and MIGD decreases by 40.3%, 45.9%, and 51.9% in the three scenarios. This demonstrates that initialization tailored to the remaining target value and makespan objectives provides high-quality starting solutions and the resulting PF approximation.

The adaptive position update further improves the performance in the scenarios. From $MOCSA_{MH}$ to $MOCSA_{MHA}$, MHV increases by 4.2% in S_2 and 9.6% in S_3, and MIGD decreases by 10.9% and 20.1%, respectively. In S_1, $MOCSA_{MHA}$ is slightly worse than $MOCSA_{MH}$, this may suggest that the smaller search space already benefits sufficiently from heuristic initialization and that additional exploratory movement may introduce minor fluctuations. The stagnation and elite guided perturbation in $MOCSA_{MHAP}$ then

improves both indicators in all scenarios, showing that local reallocation around stagnated and elite solutions is effective for refining PF approximations.

Finally, the event-aware change response provides smaller but generally favorable improvements over $\text{MOCSA}_{\text{MHAP}}$. MDCSATA obtains the best MIGD at all three scales and the best MHV in S_1 and S_3; its MHV in S_2 is only 0.2% lower than that of $\text{MOCSA}_{\text{MHAP}}$. This indicates that the event-aware change response mainly stabilizes adaptation across changes rather than serving as the dominant source of improvement.

Overall, the ablation results support that all proposed components contribute to the performance of MDCSATA, validating their effectiveness.

5.2.3. Parameter Sensitivity Analysis

To evaluate the robustness of MDCSATA to the CSA search control parameters, a full-factorial sensitivity analysis is performed using $AP \in \{0.05, 0.20\}$, $fl \in \{1.5, 2.5\}$, and $fl_{\min} \in \{1.0, 2.0\}$. The resulting eight combinations are evaluated under the same settings as in Section 5.1. This design examines both the marginal influence of each parameter and whether their effects depend on the other parameter settings. In addition, the sensitivity of the event-aware change response is examined using two fixed settings corresponding to the bounds of the adaptive rule: a conservative setting with $(R_{\text{keep}}, R_{\text{heur}}, R_{\text{lhs}}) = (0.6, 0.2, 0.2)$ and an aggressive setting with $(R_{\text{keep}}, R_{\text{heur}}, R_{\text{lhs}}) = (0.2, 0.5, 0.3)$. The MHV and MIGD results are shown in Tables 12–15.

The factorial results show that the effect of AP becomes clearer as the problem scale increases. The marginal differences between $AP = 0.05$ and $AP = 0.20$ are negligible in S_1, whereas $AP = 0.05$ improves average MHV by 1.3% and 4.6% and reduces average MIGD by 7.0% and 13.8% in S_2 and S_3, respectively. A large AP triggers random relocation more frequently and may disrupt well-structured allocations in larger search spaces. The selected value $AP = 0.05$ consequently provides the more reliable cross-scenario setting.

The $fl_{\min}$ parameter has a more consistent influence. Averaged over AP and fl, $fl_{\min} = 1.0$ improves MHV by 1.4%, 2.4%, and 4.3% and reduces MIGD by 7.1%, 8.3%, and 14.5% from S_1 to S_3. This suggests that an excessively large minimum movement can force substantial changes even when a crow should exploit a promising position. In contrast, the marginal influence of fl is small: $fl = 1.5$ is slightly better on average in S_1 and S_2, while $fl = 2.5$ is slightly better in S_3. The default combination $(0.05, 1.5, 1.0)$ gives the best MIGD in S_1, whereas $(0.05, 2.5, 1.0)$ gives the best MHV and MIGD in S_2 and S_3. Thus, the default $fl = 1.5$ is retained as a conservative cross-scenario setting, while a larger fl can assist exploration in larger search spaces.

Table 12. MHV results of the search parameter sensitivity analysis.

$(AP, fl, fl_{\min})$	S_1	S_2	S_3
$(0.20, 1.5, 1.0)$	$2.284 \times 10^5 \pm 1.504 \times 10^4$	$4.510 \times 10^5 \pm 2.740 \times 10^4$	$5.424 \times 10^5 \pm 4.416 \times 10^4$
$(0.20, 1.5, 2.0)$	$2.238 \times 10^5 \pm 1.661 \times 10^4$	$4.338 \times 10^5 \pm 3.191 \times 10^4$	$5.341 \times 10^5 \pm 4.493 \times 10^4$
$(0.20, 2.5, 1.0)$	$\mathbf{2.285 \times 10^5 \pm 1.300 \times 10^4}$	$4.468 \times 10^5 \pm 2.326 \times 10^4$	$5.628 \times 10^5 \pm 5.138 \times 10^4$
$(0.20, 2.5, 2.0)$	$2.211 \times 10^5 \pm 1.520 \times 10^4$	$4.318 \times 10^5 \pm 2.378 \times 10^4$	$5.267 \times 10^5 \pm 4.068 \times 10^4$
$(0.05, 1.5, 1.0)$	$2.279 \times 10^5 \pm 1.626 \times 10^4$	$4.453 \times 10^5 \pm 2.491 \times 10^4$	$5.759 \times 10^5 \pm 4.453 \times 10^4$
$(0.05, 1.5, 2.0)$	$2.227 \times 10^5 \pm 1.517 \times 10^4$	$4.489 \times 10^5 \pm 3.080 \times 10^4$	$5.594 \times 10^5 \pm \mathbf{3.996 \times 10^4}$
$(0.05, 2.5, 1.0)$	$2.229 \times 10^5 \pm 1.519 \times 10^4$	$\mathbf{4.529 \times 10^5 \pm 2.235 \times 10^4}$	$\mathbf{5.821 \times 10^5} \pm 4.011 \times 10^4$
$(0.05, 2.5, 2.0)$	$2.280 \times 10^5 \pm 1.505 \times 10^4$	$4.396 \times 10^5 \pm 2.655 \times 10^4$	$5.493 \times 10^5 \pm 5.173 \times 10^4$

Note: Bold denotes the best result.

Table 13. MIGD results of the search parameter sensitivity analysis.

$(AP, fl, fl_{\min})$	S_1	S_2	S_3
$(0.20, 1.5, 1.0)$	$8.177 \times 10^1 \pm 2.712 \times 10^1$	$8.316 \times 10^1 \pm 1.411 \times 10^1$	$1.519 \times 10^2 \pm 3.575 \times 10^1$
$(0.20, 1.5, 2.0)$	$8.711 \times 10^1 \pm 2.771 \times 10^1$	$9.437 \times 10^1 \pm 1.936 \times 10^1$	$1.721 \times 10^2 \pm 3.901 \times 10^1$
$(0.20, 2.5, 1.0)$	$8.394 \times 10^1 \pm \mathbf{2.050 \times 10^1}$	$8.735 \times 10^1 \pm 1.626 \times 10^1$	$1.426 \times 10^2 \pm 3.554 \times 10^1$
$(0.20, 2.5, 2.0)$	$9.006 \times 10^1 \pm 2.584 \times 10^1$	$9.668 \times 10^1 \pm 2.072 \times 10^1$	$1.736 \times 10^2 \pm 3.091 \times 10^1$
$(0.05, 1.5, 1.0)$	$\mathbf{8.010 \times 10^1} \pm 2.285 \times 10^1$	$8.342 \times 10^1 \pm 1.521 \times 10^1$	$1.297 \times 10^2 \pm 2.699 \times 10^1$
$(0.05, 1.5, 2.0)$	$8.877 \times 10^1 \pm 2.495 \times 10^1$	$8.497 \times 10^1 \pm 1.744 \times 10^1$	$1.448 \times 10^2 \pm 3.437 \times 10^1$
$(0.05, 2.5, 1.0)$	$8.324 \times 10^1 \pm 2.603 \times 10^1$	$\mathbf{7.986 \times 10^1} \pm \mathbf{1.278 \times 10^1}$	$\mathbf{1.252 \times 10^2} \pm \mathbf{2.500 \times 10^1}$
$(0.05, 2.5, 2.0)$	$8.812 \times 10^1 \pm 2.433 \times 10^1$	$8.797 \times 10^1 \pm 1.818 \times 10^1$	$1.524 \times 10^2 \pm 4.103 \times 10^1$

Note: Bold denotes the best result.

Table 14. MHV results of the event-response sensitivity analysis.

Response Setting	S_1	S_2	S_3
Adaptive	$\mathbf{2.279 \times 10^5} \pm 1.626 \times 10^4$	$4.453 \times 10^5 \pm 2.491 \times 10^4$	$\mathbf{5.759 \times 10^5} \pm \mathbf{4.453 \times 10^4}$
Conservative	$2.270 \times 10^5 \pm \mathbf{1.588 \times 10^4}$	$4.435 \times 10^5 \pm 2.771 \times 10^4$	$5.692 \times 10^5 \pm 4.782 \times 10^4$
Aggressive	$2.278 \times 10^5 \pm 1.625 \times 10^4$	$\mathbf{4.463 \times 10^5} \pm \mathbf{2.476 \times 10^4}$	$5.736 \times 10^5 \pm 4.512 \times 10^4$

Note: Bold denotes the best result.

The event-response sensitivity results in Tables 14 and 15 further show that the δ-based adaptive response is stable. It obtains the best MHV and MIGD in S_1 and S_3, while the aggressive fixed response is slightly better in S_2. The small gaps indicate that no fixed response intensity dominates across all scenarios. The adaptive rule therefore provides a robust compromise by adjusting regeneration according to the estimated severity of each environmental change.

Overall, the sensitivity study supports the selected parameter configuration and shows that the adaptive position update and the δ-based response remain stable under moderate parameter variations.

Table 15. MIGD results of the event-response sensitivity analysis.

Response Setting	S_1	S_2	S_3
Adaptive	$\mathbf{8.010 \times 10^1} \pm \mathbf{2.285 \times 10^1}$	$8.342 \times 10^1 \pm 1.521 \times 10^1$	$\mathbf{1.297 \times 10^2} \pm \mathbf{2.699 \times 10^1}$
Conservative	$8.274 \times 10^1 \pm 2.402 \times 10^1$	$8.324 \times 10^1 \pm 1.832 \times 10^1$	$1.337 \times 10^2 \pm 3.110 \times 10^1$
Aggressive	$8.062 \times 10^1 \pm 2.341 \times 10^1$	$\mathbf{8.198 \times 10^1} \pm \mathbf{1.428 \times 10^1}$	$1.309 \times 10^2 \pm 3.021 \times 10^1$

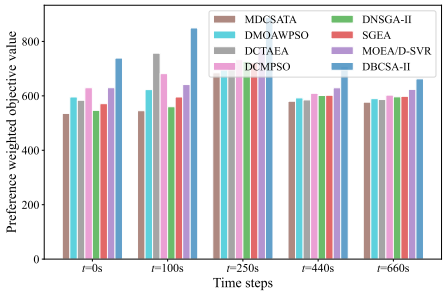
Note: Bold denotes the best result.

5.2.4. Preference-Weighted Solution Quality

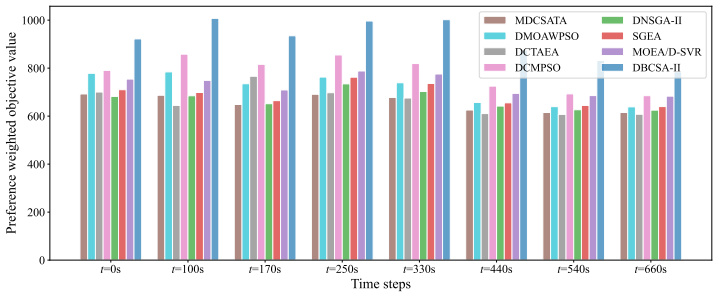
Figure 4 presents the preference-weighted objective value of the executed solution at each environmental time step, selected under the decision-maker's preference weights. Across all three scenarios, MDCSATA delivers the lowest or near-lowest weighted objective value at most time steps, reflecting well-distributed PFs.

DCTAEA and DNSGA-II are the most competitive baselines and occasionally edge ahead of MDCSATA at certain time steps, which is consistent with their MHV and MIGD results. In S_1, MDCSATA achieves a 3.56% improvement over DNSGA-II and a 9.05% improvement over DCTAEA across all time steps. In S_2, MDCSATA holds a 1.09% improvement over DCTAEA and a 1.81% improvement over DNSGA-II. In S_3, MDCSATA attains a 1.70% improvement over DNSGA-II and a 3.79% improvement over DCTAEA. This overall advantage indicates that MDCSATA sustains both front diversity and convergence more effectively as the problem environment evolves, whereas the competitors tend to lose solution quality.

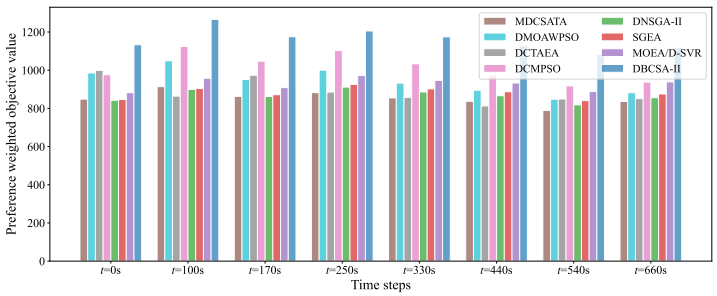
SGEA also performs well among the algorithms and occasionally matches MDCSATA, but its solution quality fluctuates across time steps and rarely surpasses the top two. The remaining algorithms operate at larger disadvantages, suggesting that their PFs are either poorly converged or insufficiently diverse to provide strong candidates under the given preference weights. DMOAWPSO trails MDCSATA by 5.63% in S_1, 8.41% in S_2, and 9.53% in S_3. Against DBCSA-II, the weakest baseline, the improvement of MDCSATA reaches 24.05% in S_1, 28.58% in S_2, and 26.48% in S_3. These consistent margins indicate that MDCSATA not only produces high-quality PF approximations but also translates this advantage into superior executed-solution quality under shifting decision-maker preferences.



(**a**) Weighted objective values of S_1.



(**b**) Weighted objective values of S_2.



(**c**) Weighted objective values of S_3.

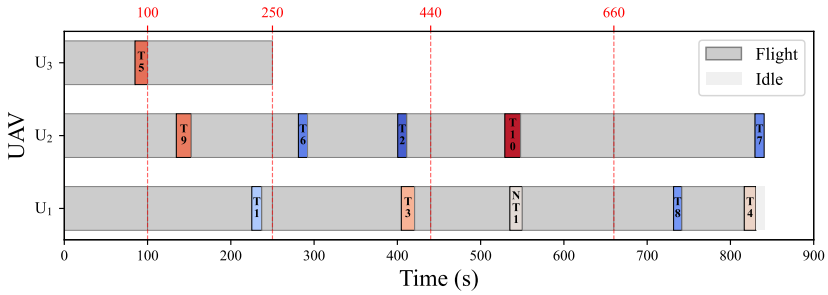
Figure 4. Preference weighted objective value of the executed solution of the algorithms across different scenarios.

5.2.5. Task Allocation Visualization

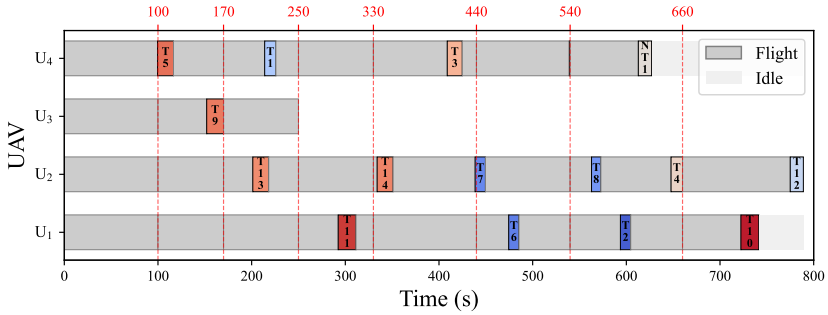
Figure 5 presents the Gantt chart of MDCSATA's execution schedule across the scenarios, and Figures 6, 7, and 8 show the per-time step spatial allocation maps for S_1, S_2, and S_3, respectively. Target colors are mapped from observational values using a cool-to-warm scale. For example, T_2 with value 52 appears blue, NT_1 with value 76 appears light gray, and T_{10} with value 99 appears red.

These visualizations highlight several advantages of the proposed model formulation and encoding design. Three validity properties are consistently maintained across all dy-

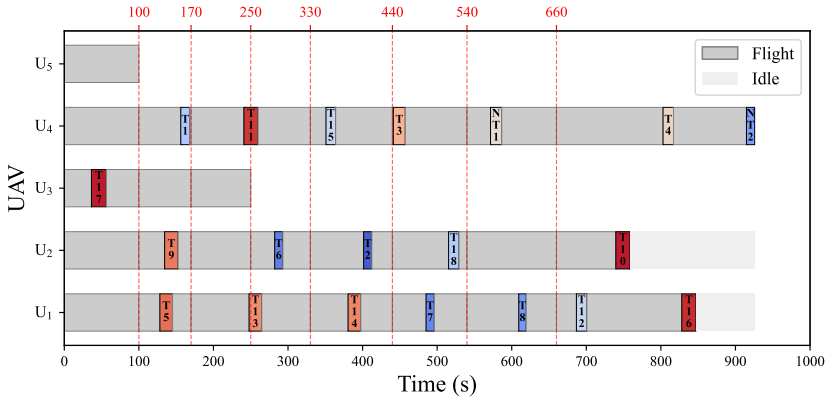
namic events. First, ongoing tasks are never interrupted, as the random-key encoding locks in-progress assignments to the executing UAV, preventing mid-task reassignment. Second, completed targets are never re-allocated, because the variable-length encoding grows monotonically as new targets appear while skipping completed targets when decoding. Third, newly appeared targets are integrated within the time steps, appearing immediately in the schedule of the following re-optimization phase. These properties are guaranteed by design, without relying on post-processing repair. The spatial allocation maps further illustrate the workload redistribution behavior under dynamic events. After a UAV crash, the surviving UAVs absorb the remaining unassigned targets with minimal makespan increase, driven by the f_2-biased heuristic initialization that seeds the swarms with load-balanced assignments. After a sensor damage event, the affected UAV's reduced sensing capability $\eta_i(t)$ lowers its contribution to f_1, causing high-value targets to be naturally shifted to UAVs with better sensing capability. These adaptive behaviors emerge directly from the objective formulation and MDCSATA's framework, requiring no post-processing rules.



(**a**) Gantt chart of S_1.

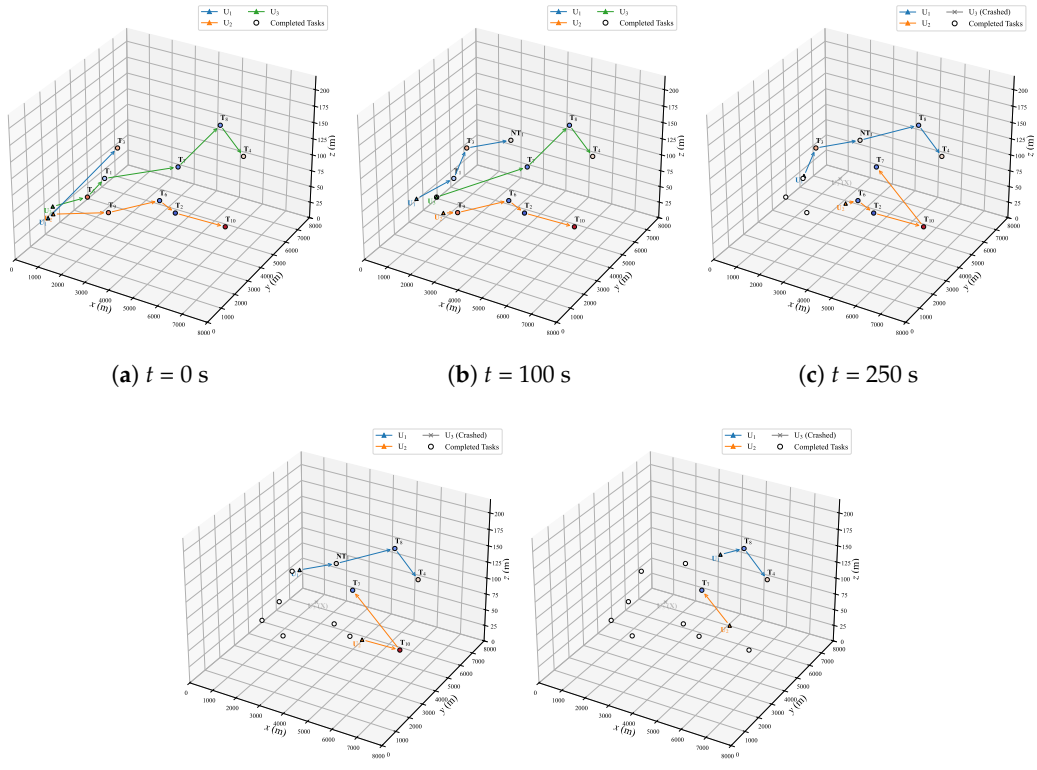


(**b**) Gantt chart of S_2.



(**c**) Gantt chart of S_3.

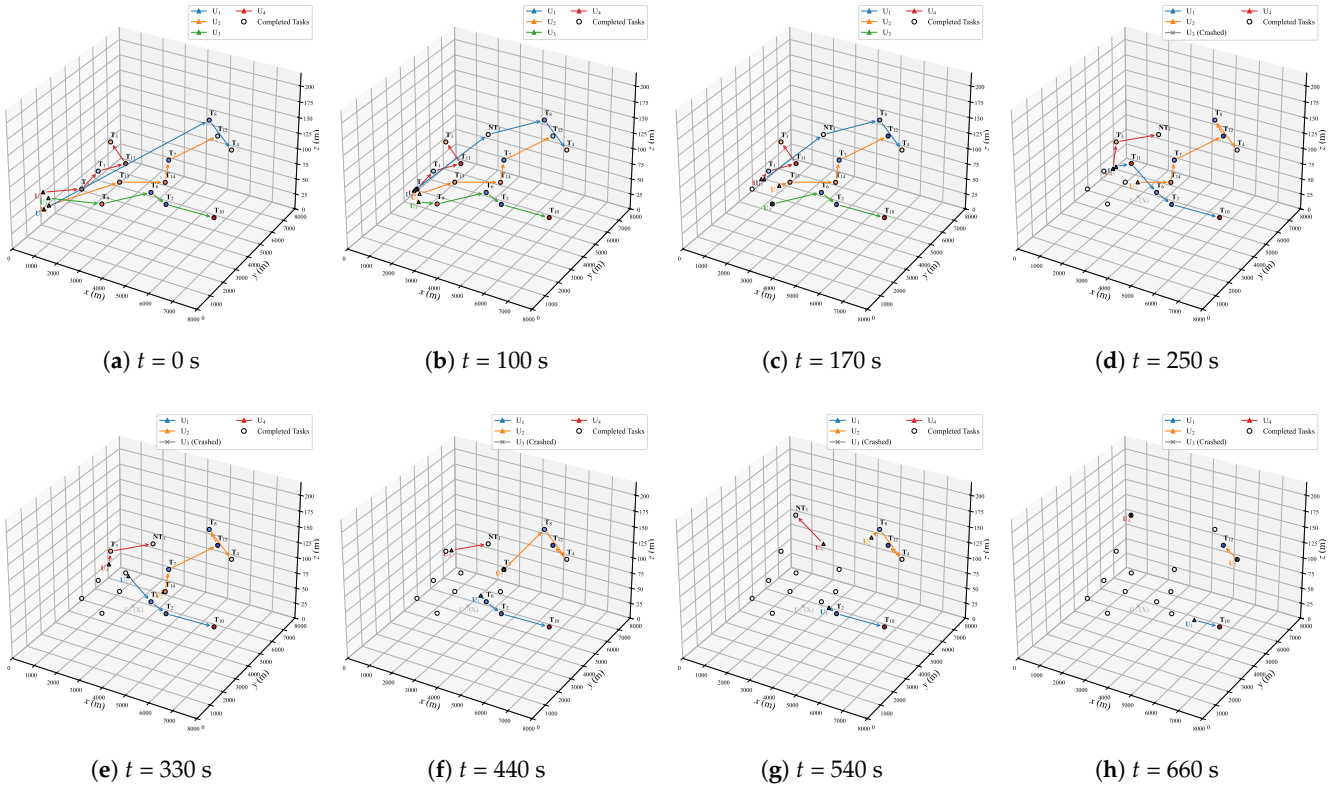
Figure 5. Gantt charts of MDCSATA's allocation scheme across different scenarios.



(**a**) t = 0 s (**b**) t = 100 s (**c**) t = 250 s

(**d**) t = 440 s (**e**) t = 660 s

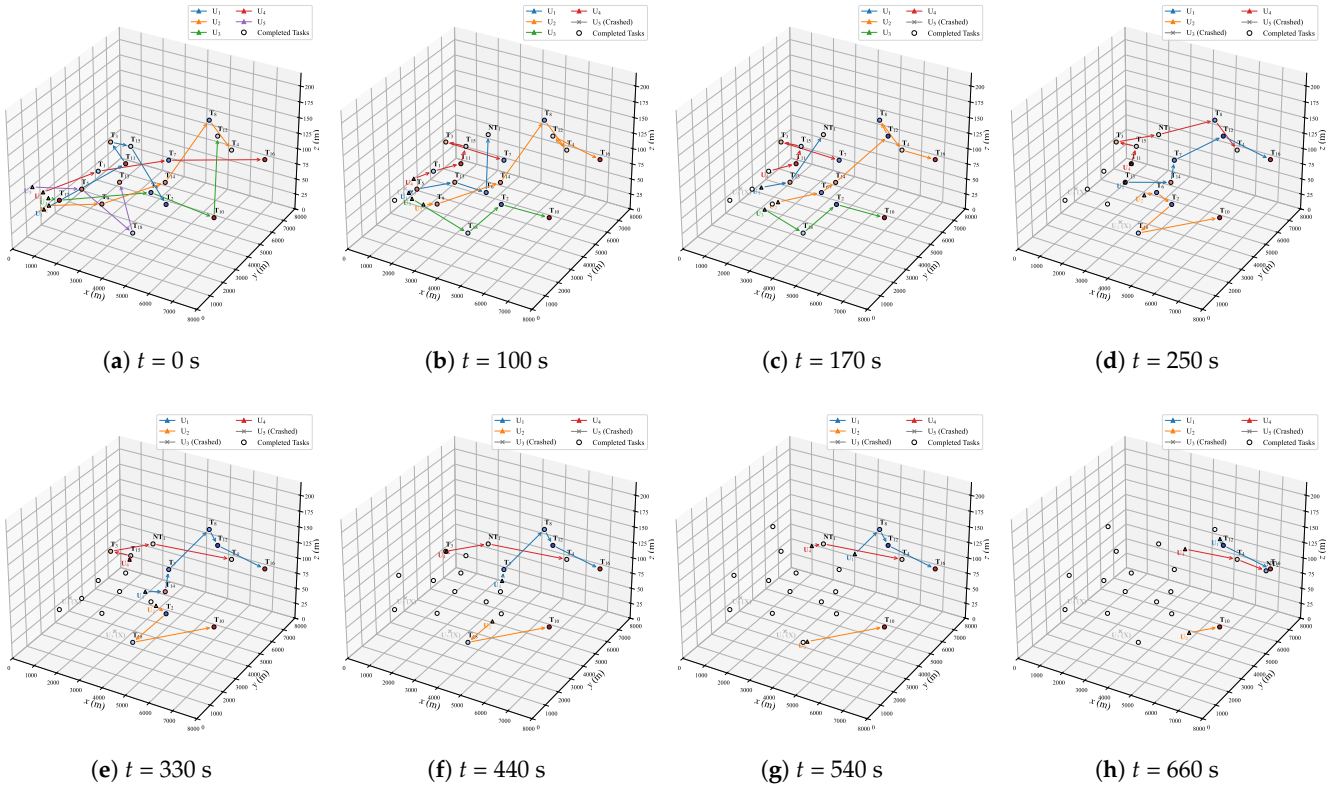
Figure 6. The allocation results of MDCSATA in S_1.



(**a**) t = 0 s (**b**) t = 100 s (**c**) t = 170 s (**d**) t = 250 s

(**e**) t = 330 s (**f**) t = 440 s (**g**) t = 540 s (**h**) t = 660 s

Figure 7. The allocation results of MDCSATA in S_2.



(**a**) t = 0 s (**b**) t = 100 s (**c**) t = 170 s (**d**) t = 250 s

(**e**) t = 330 s (**f**) t = 440 s (**g**) t = 540 s (**h**) t = 660 s

Figure 8. The allocation results of MDCSATA in S_3.

5.2.6. Runtime Efficiency

Table 16 gives the average runtime of all algorithms. Benefiting from the inherently simple search mechanism of CSA, both DBCSA-II and MDCSATA require very short optimization time. Three algorithms—DBCSA-II, DCMPSO, and MDCSATA—form a distinctly faster group across all scenarios. Among them, DBCSA-II and DCMPSO rank first or second, yet both exhibit severely degraded MHV and MIGD; in S_3, DBCSA-II fails to maintain meaningful PF tracking. MDCSATA ranks third in speed, with its additional overhead relative to the other two attributable to the integrated strategies, which is consistent with the complexity analysis in Section 4.7. Despite this, MDCSATA achieves the best MHV and MIGD, striking a favorable balance between solution quality and efficiency. The remaining algorithms require obviously more runtime than the above group, with SGEA incurring overhead roughly 30 times that of MDCSATA, yet their allocation quality does not exceed that of MDCSATA. These results indicate that MDCSATA provides a highly effective trade-off between optimization performance and computational cost, making it well-suited for time-critical operations where both accuracy and responsiveness are essential.

Table 16. Average run time of algorithms (unit: seconds).

Scenario	MDCSATA	DMOAWPSO	DCTAEA	DCMPSO	DNSGA-II	SGEA	MOEA/D-SVR	DBCSA-II
S_1	0.414	1.480	2.139	0.291	1.665	19.813	1.449	**0.289**
S_2	0.792	2.823	3.824	**0.565**	2.773	31.271	3.080	0.574
S_3	0.941	2.826	3.521	0.683	2.999	28.156	3.429	**0.641**

Note: Results in bold font represent the least average runtime.

In summary, MDCSATA consistently surpasses all seven comparison algorithms across the three scenarios on both MHV and MIGD, while maintaining third runtime. The advantage confirms that the integrated strategies deliver rapid PF tracking in the

dynamically changing problem environment, which validates MDCSATA's effectiveness in multi-UAV DTA for time-critical missions.

6. Conclusions

This paper tackled cooperative task allocation for UAV swarms in dynamic scenarios. Existing methods often fail to maintain consistency between allocation decisions and actual operational states, consider only limited classes of dynamic events, and still leave room for performance improvement. To overcome these, we formulated a DMOP with a state inheritance mechanism that continuously synchronizes allocation with actual UAV and target states. A comprehensive event model encompassing platform failures, sensor degradation, target dynamics, weather deterioration, and decision-maker preference shifts was built across three observation scenarios.

To solve this DMOP, we proposed MDCSATA. The algorithm employs a random-key encoding that maps discrete assignments to a continuous space and enforces task continuity through variable designs. Five tailored strategies are integrated: a violation-tolerant multi-swarm co-evolution preserves feasibility and diversity; objective-oriented heuristic initialization seeds the population toward each objective; an adaptive position update strengthens exploration and exploitation; a stagnation and elite guided perturbation intensifies local exploitation; and an event-aware change response quantifies severity and adaptively regenerates solutions for rapid response to dynamic events.

Extensive experiments against seven state-of-the-art DMOEAs across three scenarios show that MDCSATA achieves the best MHV and MIGD in all settings, with statistically significant improvements in 53 out of 56 pairwise comparisons. Moreover, MDCSATA maintains an acceptable runtime overhead; therefore, it is well suited for time-critical missions.

Future work can explore several directions. First, the current allocation-level model represents UAVs as point masses with a common cruise speed and Euclidean flight segments. These assumptions may produce optimistic estimates of the mission makespan, residual energy, and flight risk. Future work will couple task allocation with trajectory planning to incorporate platform-specific speeds, turning-radius constraints, obstacle avoidance, inter-UAV safe distances, detailed flight-energy models, and spatial risk fields describing threat distributions, terrain risks, and target-area risks. Their effects on makespan, energy feasibility, and survival probability will then be quantitatively evaluated. Second, extending MDCSATA to distributed coordination under intermittent communication would address practical deployment scenarios where a centralized control center is unavailable. Third, physical experiments on real UAV platforms would further validate our approach and help bridge the gap between algorithm development and real-world deployment.

Author Contributions: Conceptualization, G.L. and Y.L.; methodology, G.L. and Y.L.; software, Q.Z. and Y.L.; validation, K.L. and Y.L.; formal analysis, Q.Z. and K.L.; investigation, Q.Z. and Y.L.; resources, K.L.; data curation, Y.L.; writing, original draft preparation, G.L.; writing, review and editing, G.L. and Y.L.; supervision, K.L.; project administration, Y.L. and K.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The data will be made available on request.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Li, Y.; Qiao, G.; Popov, S.; Cui, X.; Florinsky, I.V.; Yuan, X.; Wang, L. Unmanned aerial vehicle remote sensing for antarctic research: A review of progress, current applications, and future use cases. *IEEE Geosci. Remote Sens. Mag.* **2023**, *11*, 73–93. [CrossRef]
2. Wang, P.; He, P.; Ma, C.; Niu, C.; Gao, H.; Wang, H.; Muyeen, S.M.; Zhou, D. A novel path planning approach for plant protection UAV based on DDPG and ILA optimization algorithm. *Comput. Electron. Agric.* **2025**, *239*, 111006. [CrossRef]
3. Li, H.; Chen, Y.; Liu, J.; Zhang, Z.; Zhu, H. Unmanned aircraft system applications in damage detection and service life prediction for bridges: A review. *Remote Sens.* **2022**, *14*, 4210. [CrossRef]
4. Yang, L.; Zhang, X.; Li, Z.; Li, L.; Shi, Y. A LODBO Algorithm for Multi-UAV Search and Rescue Path Planning in Disaster Areas. *Chin. J. Aeronaut.* **2025**, *38*, 103301. [CrossRef]
5. Ahmad, I. Intelligent task offloading decisions for enhanced swarm-based UAV surveillance. *IEEE Trans. Aerosp. Electron. Syst.* **2025**, 1–14. [CrossRef]
6. Li, C.; Li, G.; Liu, Y.; Zheng, Q.; Yang, G.; Liu, K.; Diao, X. Cooperative Task Allocation for Unmanned Aerial Vehicle Swarm Using Multi-Objective Multi-Population Self-Adaptive Ant Lion Optimizer. *Drones* **2025**, *9*, 733. [CrossRef]
7. Cao, P.; Lei, L.; Cai, S.; Shen, G.; Liu, X.; Wang, X.; Zhang, L.; Zhou, L.; Guizani, M. Computational intelligence algorithms for UAV swarm networking and collaboration: A comprehensive survey and future directions. *IEEE Commun. Surv. Tutor.* **2024**, *26*, 2684–2728. [CrossRef]
8. Tang, J.; Duan, H.; Lao, S. Swarm intelligence algorithms for multiple unmanned aerial vehicles collaboration: A comprehensive review. *Artif. Intell. Rev.* **2023**, *56*, 4295–4327. [CrossRef]
9. Du, Z.; Luo, C.; Min, G.; Wu, J.; Luo, C.; Pu, J.; Li, S. A Survey on Autonomous and Intelligent Swarms of Uncrewed Aerial Vehicles (UAVs). *IEEE Trans. Intell. Transp. Syst.* **2025**, *26*, 14477–14500. [CrossRef]
10. Freitas, A.; Rabbath, C.A.; Williams, C.; Lechevin, N.; Givigi, S. Multi-Unmanned Aerial Vehicles Cooperative Tactics: A Survey. *IEEE Access* **2025**, *13*, 119897–119921. [CrossRef]
11. Checker, L.; Xie, H.; Khaksar, S.; Murray, I. Systematic Review of Multi-Objective UAV Swarm Mission Planning Systems from Regulatory Perspective. *Drones* **2025**, *9*, 509. [CrossRef]
12. Peng, Q.; Wu, H.; Xue, R. Review of dynamic task allocation methods for UAV swarms oriented to ground targets. *Complex Syst. Model. Simul.* **2021**, *1*, 163–175. [CrossRef]
13. Hong, Y.; Jung, S.; Kim, S.; Cha, J. Extensions of Receding Horizon Task Assignment for Area Coverage in Dynamic Environments. *IEEE Trans. Aerosp. Electron. Syst.* **2023**, *59*, 3114–3128. [CrossRef]
14. Wang, X.; Ma, Z.; Shi, S.; Gong, H. Real-Time Task Allocation for UAV Swarms in Complex Environments via Dynamic Hierarchical Attention GNN (DHA-GNN). *IEEE Trans. Autom. Sci. Eng.* **2026**, *23*, 2573–2591. [CrossRef]
15. Han, R.; Zhang, T.; Hu, B.; Li, C. Distributed Auction-Based Adaptive Task Assignment and Re-Assignment for Multi-UAV Suppressive Jamming. *Def. Technol.* **2026**, *59*, 176–187. [CrossRef]
16. Zhang, H.; Liang, Z.; Liu, H.; Wang, R.; Liu, Y. Ensemble framework by using nature inspired algorithms for the early-stage forest fire rescue—A case study of dynamic optimization problems. *Eng. Appl. Artif. Intell.* **2020**, *90*, 103517. [CrossRef]
17. Deng, M.; Yao, Z.; Li, X.; Wang, H.; Nallanathan, A.; Zhang, Z. Dynamic multi-objective AWPSO in DT-assisted UAV cooperative task assignment. *IEEE J. Sel. Areas Commun.* **2023**, *41*, 3444–3460. [CrossRef]
18. Zhang, X.; Zheng, J.; Su, T.; Ding, M.; Liu, H. An effective dynamic constrained two-archive evolutionary algorithm for cooperative search-track mission planning by UAV swarms in air intelligent transportation. *IEEE Trans. Intell. Transp. Syst.* **2024**, *25*, 944–958. [CrossRef]
19. Wang, S.; Qiao, P.; Yue, Q.; Xu, Z.; Shang, Q. Research on dynamic particle swarm optimization for multi-objective reconnaissance task allocation of UAVs. *Drones* **2025**, *9*, 556. [CrossRef]
20. Huang, H.; Jiang, Z.; Yan, T.; Bai, Y. Dynamic Task Allocation for Heterogeneous Multi-UAVs in Uncertain Environments Based on 4DI-GWO Algorithm. *Drones* **2024**, *8*, 236. [CrossRef]
21. Jiang, S.; Zou, J.; Yang, S.; Yao, X. Evolutionary dynamic multi-objective optimisation: A survey. *ACM Comput. Surv.* **2023**, *55*, 76. [CrossRef]
22. Meraihi, Y.; Gabis, A.B.; Ramdane-Cherif, A.; Acheli, D. A comprehensive survey of crow search algorithm and its applications. *Artif. Intell. Rev.* **2021**, *54*, 2669–2716. [CrossRef] [PubMed]
23. Monga, P.; Sharma, M.; Sharma, S.K. A comprehensive meta-analysis of emerging swarm intelligent computing techniques and their research trend. *J. King Saud Univ. Comput. Inf. Sci.* **2022**, *34*, 9622–9643. [CrossRef]
24. Askarzadeh, A. A novel metaheuristic method for solving constrained engineering optimization problems: Crow search algorithm. *Comput. Struct.* **2016**, *169*, 1–12. [CrossRef]
25. Nobahari, H.; Bighashdel, A. MOCSA: A multi-objective crow search algorithm for multi-objective optimization. In Proceedings of the 2017 2nd Conference on Swarm Intelligence and Evolutionary Computation (CSIEC), Kerman, Iran, 7–9 March 2017; pp. 60–65. [CrossRef]

26. Jiang, H.; Yang, Y.; Wan, Q.; Dong, Y. Feature selection based on dynamic crow search algorithm for high-dimensional data classification. *Expert Syst. Appl.* **2024**, *250*, 123871. [CrossRef]
27. Aboud, A.; Rokbani, N.; Neji, B.; Al Barakeh, Z.; Mirjalili, S.; Alimi, A.M. A distributed bi-behaviors crow search algorithm for dynamic multi-objective optimization and many-objective optimization problems. *Appl. Sci.* **2022**, *12*, 9627. [CrossRef]
28. Li, G.S.; Liu, Y.; Zheng, Q.B.; Yang, G.L.; Liu, K.; Wang, Q.; Diao, X.C. Review on multi-sensor data fusion research for unmanned aerial vehicles. *J. Softw.* **2025**, *36*, 1881–1905. [CrossRef]
29. Bean, J.C. Genetic Algorithms and Random Keys for Sequencing and Optimization. *ORSA J. Comput.* **1994**, *6*, 154–160. [CrossRef]
30. Wang, J.f.; Jia, G.w.; Lin, J.c.; Hou, Z.x. Cooperative task allocation for heterogeneous multi-UAV using multi-objective optimization algorithm. *J. Cent. South Univ.* **2020**, *27*, 432–448. [CrossRef]
31. Zhan, Z.H.; Li, J.; Cao, J.; Zhang, J.; Chung, H.S.H.; Shi, Y.H. Multiple populations for multiple objectives: A coevolutionary technique for solving multiobjective optimization problems. *IEEE Trans. Cybern.* **2013**, *43*, 445–463. [CrossRef] [PubMed]
32. Ma, H.; Shen, S.; Yu, M.; Yang, Z.; Fei, M.; Zhou, H. Multi-population techniques in nature inspired optimization algorithms: A comprehensive survey. *Swarm Evol. Comput.* **2019**, *44*, 365–387. [CrossRef]
33. Tian, Y.; Si, L.; Zhang, X.; Cheng, R.; He, C.; Tan, K.C.; Jin, Y. Evolutionary large-scale multi-objective optimization: A survey. *ACM Comput. Surv.* **2022**, *54*, 174–207. [CrossRef]
34. Deb, K.; Pratap, A.; Agarwal, S.; Meyarivan, T. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Trans. Evol. Comput.* **2002**, *6*, 182–197. [CrossRef]
35. Escobar-Cuevas, H.; Cuevas, E.; Avila, K.; Avalos, O. An Advanced Initialization Technique for Metaheuristic Optimization: A Fusion of Latin Hypercube Sampling and Evolutionary Behaviors. *Comput. Appl. Math.* **2024**, *43*, 234. [CrossRef]
36. Necira, A.; Naimi, D.; Salhi, A.; Salhi, S.; Menani, S. Dynamic crow search algorithm based on adaptive parameters for large-scale global optimization. *Evol. Intell.* **2022**, *15*, 2153–2169. [CrossRef]
37. Deb, K.; Udaya Bhaskara Rao, N.; Karthik, S. Dynamic multi-objective optimization and decision-making using modified NSGA-II: A case study on hydro-thermal power scheduling. In Proceedings of the 4th International Conference on Evolutionary Multi-criterion Optimization, Matsushima, Japan, 5–8 March 2007; Volume 4403, pp. 803–817. [CrossRef]
38. Jiang, S.; Yang, S. A steady-state and generational evolutionary algorithm for dynamic multiobjective optimization. *IEEE Trans. Evol. Comput.* **2017**, *21*, 65–82. [CrossRef]
39. Cao, L.; Xu, L.; Goodman, E.D.; Bao, C.; Zhu, S. Evolutionary dynamic multiobjective optimization assisted by a support vector regression predictor. *IEEE Trans. Evol. Comput.* **2020**, *24*, 305–319. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.